

Intelligent Drawing Cobot



Submitted By

M. Usama Zafar	2016-MC-58
Hasnain Yaseen	2016-MC-65
Abdullah Shahzad	2016-MC-75

Project Advisor:

Dr. Abbas Zulqarnain

**A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF
REQUIREMENT FOR THE DEGREE OF**

Bachelor of Mechatronics and Control Engineering

**DEPARTMENT OF MECHATRONICS & CONTROL ENGINEERING
UNIVERSITY OF ENGINEERING & TECHNOLOGY, LAHORE**

INTELLIGENT DRAWING COBOT

**THIS PROJECT IS SUBMITTED TO THE DEPARTMENT OF MECHATRONICS &
CONTROL ENGINEERING FOR THE PARTIAL FULFILLMENT OF**

Bachelors Degree

MECHATRONICS & CONTROL ENGINEERING

APPROVED BY

Internal Examiner

External Examiner

**DEPARTMENT OF MECHATRONICS & CONTROL ENGINEERING
UNIVERSITY OF ENGINEERING & TECHNOLOGY, LAHORE**

Declaration

The work cited in this report, titled “*Intelligent Drawing Cobot*” has been performed as a requirement for the completion of a bachelor’s degree in Mechatronics and Control Engineering at UET Lahore. We declare that the work was performed by us and not copied or outsourced. Furthermore, we declare that the work will not be used for commercial purposes without the prior consent of the project advisor and department.

Group member no. 1

Signature

Group member no. 2

Signature

Group member no. 3

Signature

***DEDICATED TO OUR
BELOVED, HONORABLE & RESPECTED
PARENTS & TEACHERS***

Acknowledgments

All thanks to Allah Almighty who is the most merciful and beneficial. By His grace, we have attempted technical innovation through our project. We would like to extend our heartiest and humble gratitude to our project advisor, **Dr. Abbas Zulqarnain Naqvi**, whose encouragement, guidance and support from the initial to the final level enabled us to develop an understanding of the problem statement and hence allowed us to bring our senior year project to its completion.

Most of all, we are grateful to our parents for the unfailing encouragement they have given us over the years. We are extremely thankful to the respected research officer, **Mr. Muhammad Bilal** for his guidance regarding the implementation of the required control scheme.

Regards,

Abdullah Shahzad

M. Usama Zafar

Hasnain Yaseen

Table of Contents

CHAPTER 1- INTRODUCTION	1
1.1 Introduction:	1
1.1.1 Collaborative Robot:	1
1.1.2 SCARA:	2
1.1.3 Kinematics:	2
1.1.4 Admittance Control :	2
1.1.5 Curve Fitting :	3
1.2.Motivation And Objective:.....	4
1.3.Proposed Solution:.....	4
1.4 Scope:	5
CHAPTER 2 – Literature review	6
2.1 Mechanical interaction:	6
2.2 Impedance control:	6
2.3. Admittance control:	7
2.4. Forward Kinematics:.....	7
2.5. Inverse Kinematics:	8
2.6. Robot Dynamics:	9
2.6.1.Dynamics Algorithms:	10
2.7. Jacobian:	10
2.8. Simsacpe:	11
CHAPTER 3 - COMPONENTS	13
3.1 Serpent-1 Robot:.....	13
3.1.1 DC Motor:.....	13
3.1.2 Motor Driver:	14
3.1.3 Encoders:.....	15
3.1.4 Optical Absolute Encoders:.....	15
3.1.5 Incremental Encoders:.....	15
3.1.6.Limit Switches:	16
3.2 Arduino Atmega2560:	17
3.3 Force Sensitive Resistor:	17
CHAPTER 4 - METHODOLOGY	18
4.1 Admittance Control:.....	18
4.1.1.Forward Kinematics:	20
4.1.2 Inverse Kinematics:.....	21
4.2 Curve Fitting:	21
CHAPTER 5:CONTROLLER PROGRAMMING.....	23
5.1.Arduino Working Environment:	23

5.1.1.Sketch Making:	23
5.1.2.PIN Configuration:	24
5.2Function's Description:.....	24
5.2.1 Timer function:	24
5.2.2 Sampling input function:.....	25
5.2.3 Actuation functions:	25
5.2.4 Calculation functions:	25
5.2.5 linearization functions:.....	25
5.3 Flow of the program:.....	26
5.3.1: Acquiring the Position of the manipulator:	26
5.3.2: Acquiring Sensor values:	26
5.3.3:Calculations:	26
5.3.4: Application of admittance law:	26
5.3.5: Calculating PWM:	27
5.3.6: Linearizing the output:	27
5.3.7 Position Control:	27
CHAPTER 6- SIMULATION	28
CHAPTER 7- RESULTS	32
7.1 Simulation Results:.....	32
7.2 Hardware Results:	36
7.2.1 Application of force:	36
7.2.2 Displacement by force:	37
7.2.3 Return to reference position:	37
CHAPTER 8- CONCLUSION	38
8.1.Future Enhancements:	38
8.1.1.Better Sensors:	38
8.1.2.Use of Harmonic Gear Drives:.....	38
8.1.3.Use of Image Processing:.....	38
8.2. Environment and Sustainability goals:	38
8.3.Conclusion:.....	39
CHAPTER 9-APPLICATIONS	40
9.1.Path Guiding:	40
9.2.Safety Factor:	40
9.3.Automated Assembly:	41
9.4.Automated Production Line:	41
REFERENCES.....	43
APPENDIX	44

LIST OF FIGURES	PAGE NO.
FIGURE-1.1: COLLABORATIVE ROBOT	1
FIGURE-1.2: SERPENT-1 FOR DRAWING	2
FIGURE-1.3: CONTROL SCHEME OF ADMITTANCE CONTROL	3
FIGURE-1.4: CURVE FITTED ON POINTS	3
FIGURE-1.4:PROPOSED FLOW OF PROJECT	4
FIGURE-2.1:2-LINK MANIPULTOR	8
FIGURE-2.2:PHYSICAL DOMAIN IN SIMSCAPE	12
FIGURE-2.3:MASS SPRING SYSTEM	12
FIGURE-2.4:SCHEMATIC OF MULTIDOMAIN MODEL	12
FIGURE-3.1: SERPENT-1 ROBOT	13
FIGURE-3.2: DC MOTOR	13
FIGURE-3.3: CIRCUIT DIAGRAM OF L298	14
FIGURE-3.4: DIRECTION CONTROL OF MOTOR	15
FIGURE-3.5: ABSOLUTE ENCODERS	15
FIGURE-3.6: INCREMENTAL ENCODER	15
FIGURE-3.7: SIGNAL DIAGRAM OF INCREMENTAL ENCODER	16
FIGURE-3.8: LIMIT SWITCHES	16
FIGURE-3.9: ARDUINO ATMEGA2560	17
FIGURE-3.10: FORCE SENSITIVE RESISTORS	17
FIGURE-4.1: BLOCK DIAGRAM OF ADMITTANCE CONTROL	20
FIGURE-4.2 FORWARD AND INVERSE KINEMATICS	20
FIGURE-4.3: TWO LINK MANIPULATOR	21
FIGURE-4.4: CURVE FITTING OF A CIRCLE	22
FIGURE-5.1: ARDUINO IDE	23
FIGURE-5.2:ARDUINO TIMER REGISTER	24
FIGURE-5.3:MOTOR SPEED CONTROL	25
FIGURE-5.4:PID CONTROL	27
FIGURE-6.1: SIMSCAPE MODEL OF ROBOT	28
FIGURE-6.2: TWO DOF MANIPULATOR IN SIMSCAPE	29
FIGURE-6.3: ROBOT DYNAMICS IN SIMULINK	30
FIGURE-6.4: ADMITTANCE LAW IN SIMULINK	30
FIGURE-6.5: COMPLETE SIMULINK MODEL OF ADMITTANCE CONTROL	31
FIGURE-7.1: SIMULATION RESULT OF ADMITTANCE CONTROL	32
FIGURE-7.2: FORCES IN X AND Y DIRECTION	32
FIGURE-7.3: CIRCLES DRAWN USING A.CONTROLAND CURVE FITTING	33
FIGURE-7.4:CIRCLE FIT	34
FIGURE-7.5:RECTANGLE FIT	35
FIGURE-7.6 APPLICATION OF FORCE	36
FIGURE-7.7 EXTEND TO REFERENCE POSITION	36
FIGURE-7.8 COME BACK TO ORIGINAL POSITION	37

FIGURE-8.1: PATH GUIDING	40
FIGURE-8.2: SAFETY LINES	41
FIGURE-8.3: AUTOMATIC ASSEMBLY LINE	42
FIGURE-8.4: AUTOMATIC PRODUCTION LINE	42

LIST OF TABLES

PAGE NO.

TABLE-1

24

NOMENCLATURE:

LIST OF SYMBOLS	MEANINGS
L1	LENGTH OF the FIRST LINK
L2	LENGTH OF the SECOND LINK
R1	DISTANCE OF CENTRE OF MASS OF LINK-1 FROM ORIGIN
R2	DISTANCE OF CENTRE OF MASS OF LINK-2 FROM THE END OF LINK-1
θ_1	ANGLE OF LINK-1 WITH X-AXIS
θ_2	ANGLE OF LINK-2 CONCERNING θ_1

LIST OF ABBREVIATIONS	MEANINGS
SCARA	SELECTIVE COMPLIANCE ASSEMBLY ROBOT ARM
COBOT	COLLABORATIVE ROBOT
PWM	PULSE WIDTH MODULATION
DOF	DEGREES OF FREEDOM

Abstract

Intelligence in the field of robotics is the state of the idea which allows the robot to make decisions on specific tasks. In the era of collaborative robots, human input to a robot needs to be cross-checked by robot intelligence. This report is basically about a drawing robot that can learn with the aid of admittance control and reproduce the intended shape by Curve Fitting. The main aim of this project is to present an idea of Curve fitting in the robot task (drawing of analytical shapes) so that robots can easily generate the intended shape.

CHAPTER 1- INTRODUCTION

1.1 INTRODUCTION:

Human-Robot interaction is one of the advanced fields in Modern World Robotics. This interaction is achieved by the approach of impedance and admittance control. With the addition of Artificial intelligence, these controls robots can take decisions to human interface. Recent efforts have been made on the use of Intelligence in Robot Manipulation. Recently Chinese are making the drawing with the use of this State-of-the-Art Technology [1]. It can be employed to make a drawing robot that can learn with the Human-Robot Interface.

Intelligent Drawing Cobot is a robot that is capable of drawing basic shapes through Template Matching and collaboration with a human. It is the Serpent-I robot of SCARA configuration with its axis of movement in the X-Y plane. It is a 2-link manipulator with two degrees of freedom. The collaboration with human will be done through Admittance Control. In Admittance Control, user-applied force is taken from the end-effector as input while the manipulator produces motion in the direction of the application of the force as an output. This motion is directly proportional to the exerted force. The robot can learn and redraw the intended shape.

1.1.1 Collaborative Robot:

Collaborative Robot is one of the modern world Robots that are used for safe human-robot interaction. Scientists are working on the safety of the environment in the presence of robots which can be ensured by this robot. The other main advantage of this type of robot is that it could assist you as well as resist you for learning and rigid behavior respectively. We can train such a robot to safely do the task that is cumbersome for humans by using this modern technology. In the present era, it is used for training heavy systems. The main feature that is added by this robot is that it ensures the safety of humans to Giant Robots.



Figure 1.1 Collaborative Robot

1.1.2 SCARA:

The given robot is of SCARA configuration which stands for Selective Compliance Articulated Robot Arm. This type of robot is used for pick and place application for an industrial setup where high-speed work is demanded and high precision is required. We are using the serpent-1 type of SCARA manipulator that is used for drawing purposes. The pick and place feature of this robot is replaced by the drawing end effector. The goal of this conversion is to fluently draw the desired shapes without any disturbances. There are total four degrees of freedom in the SCARA Configuration while for the simple operation of Drawing there are the only degree of freedom required so we are just eliminating the up and down movement and the motion of the end effector which is fixed and an adjustable surface is added which is most probably acrylic and the whiteboard is attached on the surface and on that the manipulator with Erasable Marker attached will move.

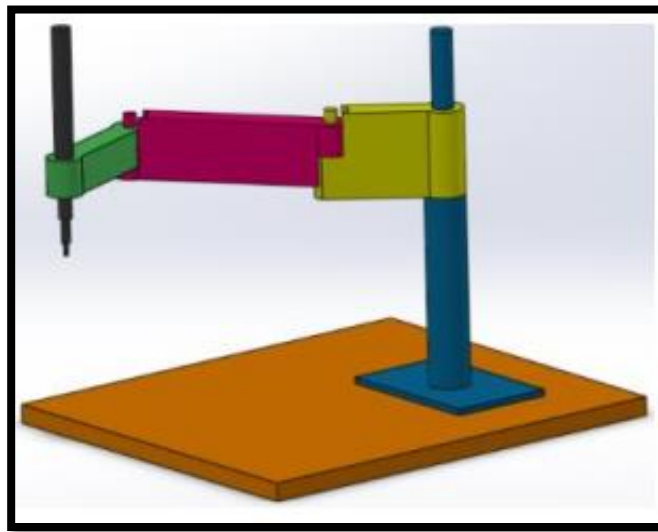


Figure 1.2 Serpent -1 for Drawing

1.1.3 Kinematics:

Kinematics is the branch of Robotics that deals with the conversion between the Angular space and the Cartesian Space. It is further split into two types.

- Forward Kinematics
- Inverse Kinematics

1.1.4 Admittance Control :

Admittance Control is one of the many techniques used for human-robot interaction. Force on the end effector is applied on the manipulator end-effector while the resulting output has been recorded in the form of the displacement from the position. The assistance or resistive behavior is decided whether you choose the reference updating or you want to go with the constant reference such that after displacing it will come back to its original position. The behavior of spring as well as of a

damper is added by incorporating the stiffness matrix as well dissipative constant. The value of this constant will give you the required behavior. Larger the value of these constants, the stiffer the robot becomes.

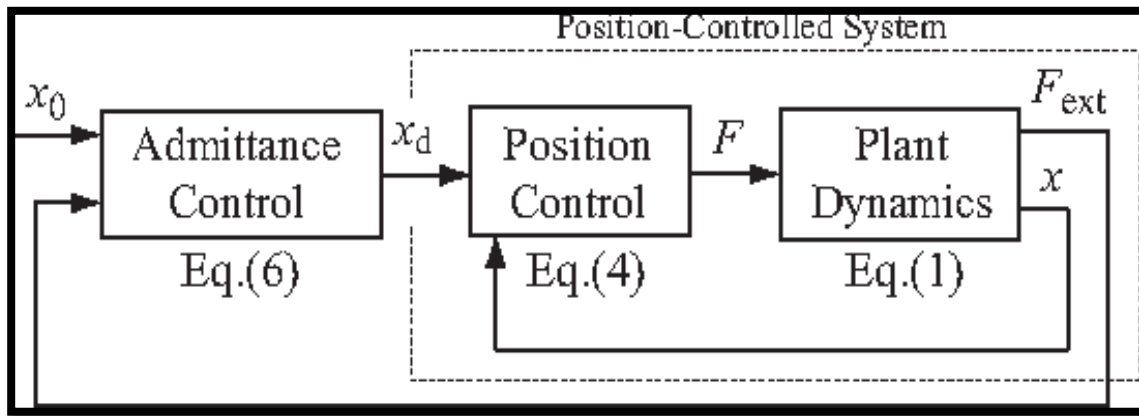


Figure 1.3 Control scheme of admittance Control

1.1.5 Curve Fitting :

The intelligence included in our project is defined in terms of curve fitting. This is the main tool that predicts not only the future values but also provides the criteria to check our motion model. It also checks the uncertainty in our reading as well as predicts the future. For its vast applications, it is mainly used in many AI algorithm to incorporate the noise of sensors as well as incorporating the uncertainty in motion

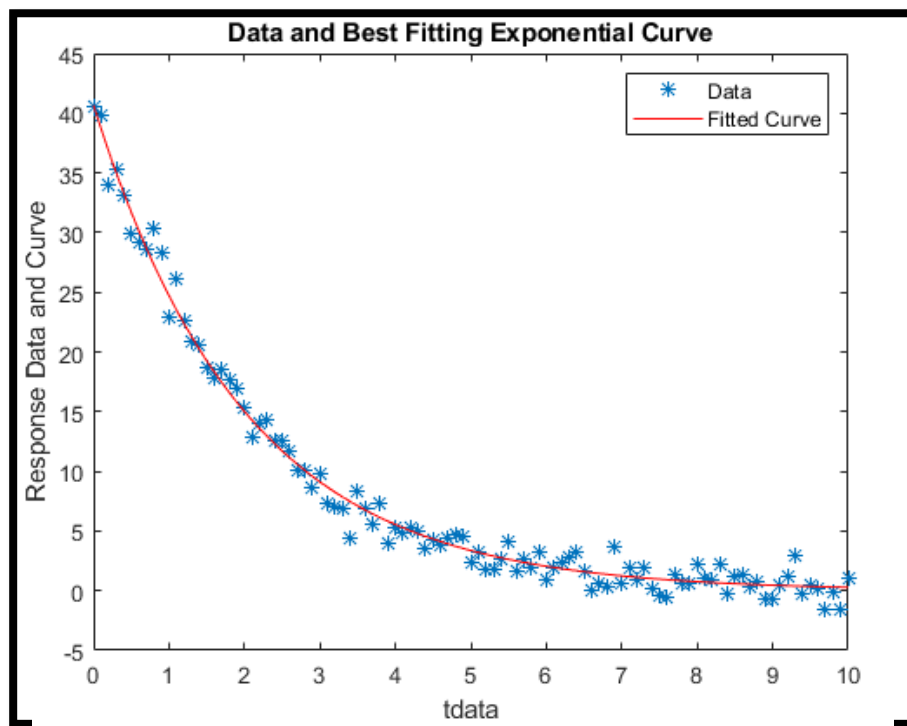


Figure 1.4 Curve fitted on points

1.2.MOTIVATION AND OBJECTIVE:

- One of the goals behind doing this advanced Research project is to connect with the modern world robots likely used for a Safe human-robot environment and the main aim to do the high-level task by using cheap but reliable hardware that can serve as a product to the low-level industry. The process of Curve fitting is adding the intelligence and future prediction to the safe environment. The process of curve fitting can greatly reduce insufficient training and forward the best result.
- To develop a collaborative drawing robot that can learn and reproduce intended shape collaboratively drawn by the user. The main objective behind this project is the learning of robots and their friendly interaction with humans working around them. Through learning, robots can be used as task differentiator. They will be just given a trajectory to perform a specific task, they will learn that path and then follow it to perform that task.
- The purpose is to make a hand-tool for the layman that can be trained and used for the repeated task in daily life task as well as for industrial setup.

1.3.PROPOSED SOLUTION:

We proposed a very simple approach to making a collaborative robot. In this approach, we simply turned the physical input from the user or layman to a most refined and fitted output which is then served as a corrected input for the repeated task.

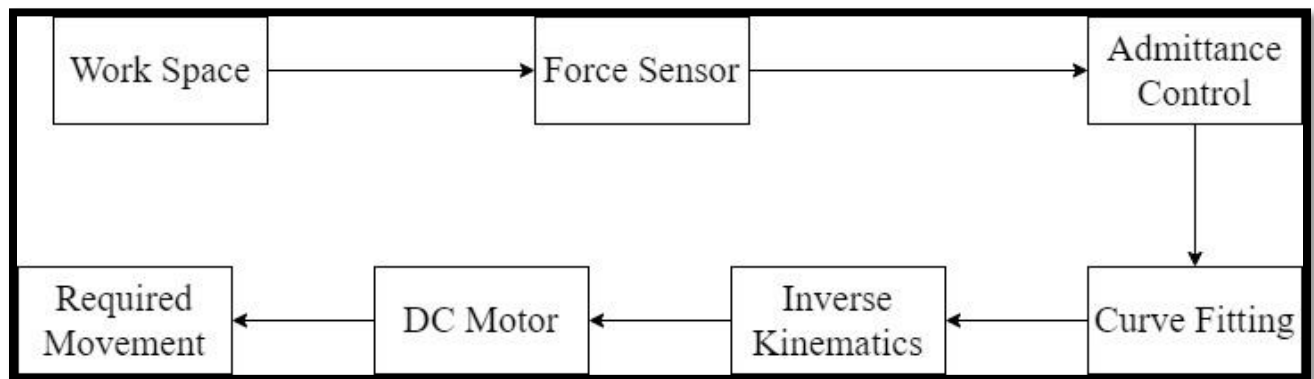


Figure 1.5 Proposed Flow of project

1.4 SCOPE:

Control of 2-link drawing manipulator to learn through waypoints and reproduce the intended analytical shape by template matching. It is a 2-link manipulator with 2 DOF. It can only draw those analytical shapes that have a mathematical equation so that, the waypoints obtained can be fitted in equations and the equation(s) that best fit those points can be found and shape(s) associated with that equation can be drawn. As this robot has 2 DOF, therefore, the drawing surface will be plain.

CHAPTER 2 – LITERATURE REVIEW

2.1 MECHANICAL INTERACTION:

The behavior of robots plays an important role in the industry. One of the most important behavior is the Mechanical Interaction of robots with objects. One of the most important applications is the interaction of robots while performing assembly tasks. If this interaction of robots is controlled properly then robots can be helpful in many other applications like the use of different tools and interaction with humans.

Mechanical interaction of robots with different objects can be controlled using sensors and controlling motor behavior. Mechanical impedance is one of the interaction control schemes in which robot dynamics are involved in controlling motors and interaction of robots. The value of mechanical impedance greatly depends on the application of the robot. For very sensitive and fragile tasks where the task can be completed by little interaction of robot, low mechanical impedance is required. For many tasks where large interaction force and stability is required, large mechanical impedance is required for stabilizing the robot. The major objective of control can be different for different applications. In the application of force-reflecting haptic display or for physiotherapy, the robot interacts with humans and that interaction should be controlled properly. The interaction of a robot with a human is controlled by mechanical interaction dynamics.

2.2 IMPEDANCE CONTROL:

The most suitable control process for controlling mechanical interaction is Impedance Control. This control scheme is the relationship between the force of contact and motion of the robot. The main aim of impedance control is to control this relationship for the completion of a particular task. The main advantage of impedance control is that it controls motion and contact forces simultaneously. Therefore, we can achieve high performance and safe environment for the interaction of humans and robots Song, Yu and Zhang[2].

Impedance control is referred to as the ratio of output effort and input flow. In the case of impedance control, input flow is the motion of manipulator while output flow is forces applied. So, the transfer function of impedance control is as follow:

$$Z(s) = \frac{F(s)}{\dot{X}(s)} \quad (2.1)$$

Above equation can also be written as:

$$Z(s) = Ms + B + \frac{K}{s} \quad (2.2)$$

Where “M” is mass, “B” is damping constant, and “K” is spring constant. From the above equation, it can be stated that the impedance control scheme is nothing but simply a mass-spring-damper

system. It is obvious from the transfer function of impedance control depends on the mass of the system, damping constant of system and spring constant.

Mechanical interaction of robots using impedance control can be controlled by varying the values of spring constant and damping constant. If large values of these constants are chosen then a large force will result in small motion and the system will remain stable even if the interaction is of high level like welding of different parts using robots.

2.3. ADMITTANCE CONTROL:

Admittance control is another control scheme for the mechanical interaction of robots. It is very much close to impedance control. The only difference between impedance and admittance control is input flow and output effort. In admittance control, input flow is contact force and output effort is the motion of the manipulator. The transfer function of admittance control is:

$$G(s) = \dot{X}(s)/F(s) \quad (2.3)$$

After simplification, the expression for admittance control becomes the same as that in impedance control as given in eq. 2.2.

$$G(s) = Ms + B + \frac{K}{s} \quad (2.4)$$

From the above eq. 2.4 it can be observed that if mass, damping coefficient, and spring coefficient are kept constant then the motion of manipulator will completely depend on force applied at end-effector. And if force is kept constant then the motion will depend on the damping and spring coefficient.

2.4. FORWARD KINEMATICS:

Forward kinematics is a function that is used for calculation of the linear position of end-effector with help of angular position which is taken as input while the output is the linear position. Using Forward Kinematics, the whole position of the articulated body needs to be defined for providing the function whose input is the pose. It means that the angular position of each joint in the body needs to be defined. So basically, it shows that one needs to use the robot's equations of kinematics to formulate the position of the end effector based on the specified input values for parameters of joints.

Now we consider the example of 2 link planar manipulator. In general, we use the DH or the Denavit-Hartenburg convention for finding out the position of the end effector based on provided joint parameters [1]. Joint parameters or variables refer to the angle between the links.

Consider the following diagram:

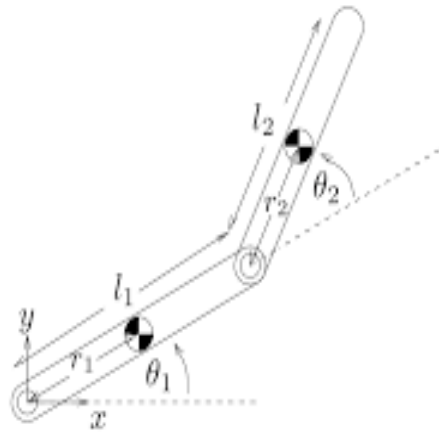


Figure 2.1 2-link planar manipulator

The DH-table for the above system is given as:

Table 1 DH-table

Link	a_i	α_i	d_i	$\Theta_{\theta i}$
1	$A1$	0	0	$\Theta_{\theta 1}$
2	$A2$	0	0	$\Theta_{\theta 2}$

In the DH method we calculate the following transformation:

$$A = \text{Rot}(z, \theta) \text{Trans}(z, d) \text{Trans}(x, a) \text{Rot}(x, \alpha) \quad (2.1)$$

The mathematical equations of Forward Kinematics for 2 link planar manipulator are:

$$X = L1 \cdot \cos(\theta_1) + L2 \cdot \cos(\theta_1 + \theta_2) \quad (2.2)$$

$$Y = L1 \cdot \sin(\theta_1) + L2 \cdot \sin(\theta_1 + \theta_2) \quad (2.3)$$

2.5. INVERSE KINEMATICS:

In the Inverse Kinematics, we use the reverse method i.e., to find joint variables when orientation and position of end-effector are provided. This procedure involves solving a set of equations. Generally, the equations are complex and non-linear, and therefore, the Inverse Kinematics procedure can become quite bulky.

We now consider our case and see how its Inverse Kinematics can be evaluated. This technique allows us to find the angles of joints θ_1 and θ_2 in terms of the end effector's position. The law of cosines provide us the following relation for theta 2:

$$\cos(\theta_2) = (x^2 + y^2 - \alpha_1^2 - \alpha_2^2) / (2 \cdot \alpha_1 \cdot \alpha_2) = D \quad (2.4)$$

And the other equation is:

$$\sin(\theta_2) = \pm(\sqrt{1-D^2}) \quad (2.5)$$

Solving for θ_2 we get:

$$\theta_2 = \text{atan}(\pm \sqrt{1-D^2}/D) \quad (2.6)$$

and finally, θ_1 can be found as :

$$\theta_1 = \text{atan}(y/x) - \text{atan}((\alpha_2 \sin(\theta_2))/(\alpha_1 + \alpha_2 \cos(\theta_2))) \quad (2.7)$$

2.6. ROBOT DYNAMICS:

As the name suggests, it is related to the relation between acceleration generated by forces and forces applied on robot. Typically, the mechanism of robot is considered as system of rigid bodies, in which case dynamics of robot is the application of dynamics of rigid bodies to robots. Just like kinematics, dynamics also has two problems:

1. Forward Dynamics

2. Inverse Dynamics

The equation of motion for a mechanism of robots is written as:

$$\tau = H(q)\ddot{q} + c(q, \dot{q}, f_{ext}) \quad (2.8)$$

In above equation, $\ddot{q}$, q , τ and $\dot{q}$ are acceleration's, joint positions, force variables, velocity's vectors, respectively, and they are known as vectors of joint-space acceleration, position, force, and velocity [1]. Each is a coordinate vector with n dimensions, where n are the mechanism's joint variables. The variables of forces which are defined as τ and f_{ext} where f_{ext} is the power that is delivered by τ to the system. So, τ and $\dot{q}$ are qualified as variables of force and velocity.

The equation of motion can also be written in two other ways that are helpful for systems of robot controls. The first one is:

$$\tau = H(q)\ddot{q} + C(q, \dot{q})\dot{q} + \tau_g(q) + J(q)^T f_{ext} \quad (2.9)$$

The term $C\dot{q}$ is composed of centrifugal and Coriolis forces, and it is the product of velocity vector and matrix C . The vector τ_g includes the effect of gravitational forces, and the last term in the above equation tells about the impact of external forces on robot dynamics. The matrix J is the Jacobian or velocity matrix for end-effector. It satisfies the equation:

$$\dot{V}_e = J\dot{q} \quad (2.10)$$

The operational-space formulation is another way for the expression of the equation of motion. It explains the robot dynamics in the task space of the robot, and it is space in which the robot is asked to work. Typically, the orientation and position's space is called task space. The formulation of operational-space is written as:

$$\Lambda \dot{v} + \mu + \rho = f \quad (2.11)$$

The velocity of the end-effector is given as a spatial vector. The force acting on the end-effector is given as a vector f . The equation of motion's coefficients is given as Λ , μ , and ρ . Λ is known as the matrix of inertia in operational space; μ is composed of terms of centrifugal and Coriolis forces. and ρ is composed of terms of gravity. f is the addition of actuator forces' projection which are acting on joints and external forces which are applied on end-effector

2.6.1.Dynamics Algorithms:

Robot dynamics uses the following three algorithms:

- The first algorithm is the recursive Newton-Euler algorithm (RNEA). The problem of inverse dynamics is solved using this algorithm and $O(n)$ is its computational complexity.
- The second algorithm is an articulated-body algorithm (ABA). The problem of forwarding dynamics is solved using this algorithm and $O(n)$ is its computational complexity.
- The third algorithm is a composite-rigid-body algorithm (CRBA). This algorithm calculates the matrix of inertia which is in joint-space. The computational complexity of this algorithm is (On^2) . It is combined with the RNEA algorithm for calculation of "c". It is also combined with a linear equation for solving $H\ddot{q} = \tau - c$. It results in an algorithm that can solve the problem of forward dynamics whose complexity is $O(n^3)$. For large values of "n", the ABA algorithm is faster as compared to CRBA. But for small values of "n", the efficiency of both algorithms is the same.

2.7. JACOBIAN:

Jacobian is nothing but a time derivative of the kinematics equation as described above, which develops a relationship between the velocities from linear to angular space. The law of apparent work shows that a relation between the forces of joints to the resulting forces at the manipulator end effector.

2.7.1. Velocity Kinematics:

The output from the Jacobian is a system of linear equations. This linear equation relates the linear and angular velocities to the given displacement while acting as a transformation. The first three Elements correspond to the linear analysis while the other three composed of angular motion. All the matrix transform is made by the basic rules of joint displacement and offset of the links.

The inversion of this Jacobian matrix relates the linear velocities to angular velocities. The output is the angular velocity or the variable we wish to compute which is correspond to the joint function

an
it must be kept in mind that jacobian matrix size corresponds to the number of links of the manipulator.

2.7.2. Static Force Analysis:

The laws of apparent work yield a system of linear equations that produce a relationship between the resultant vector of the pull which is force is defined in terms of a wrench, that are applied to the end position of the robot termed as an end effector. If the values at that end effector are known, then a direct procedure is applied to determine the values of the angular torques at joints. however, the position near to the singularity conditions is comparatively high which is the worst condition for the smooth running of Robot while the mechanical advantage is concerned it is large near that condition.

The reverse to this problem decode the end-effector wrench related to the given torques at joints and take the jacobian matrix as an input for the computation of torque. For determining the velocity of the end effector which is the inverse problem cannot be generated due to the shortage of information. As far as the mechanical advantage is concerned it is very large near a singularity.

Jacobian Matrix computed above is one the main feature of Robot dynamics which is used not only for mechanical movement but also used in the Control of Cobot. For the smooth trajectories of a robot, this thing should be computed like in the computation of the worst scenarios like in singularity and it is also used in the transformation from the manipulator end effector to the joint forces. Thus by using these forces and torques are moved from Cartesian to angular space and vice versa.

As the Jacobian is concerned with the velocities computation, we are more concerned with the investigation of velocities. First, we used a fixed axis for the computation and then we move our procedure to then move to the different axis by the aid of the skew-symmetric matrix. By using this computation of angular velocities we can drive the linear velocities as well.

2.8. SIMSACPE:

Simscape is a tool that is used in the Simulink environment for the creation of a physical model of systems. Using this tool, physical models of systems can be generated which are based on physical connections. These models are made using block diagrams. Various mechanical, electrical, and hydraulic models can be made by using fundamental tools of Simscape. With the help of add-ons of Simscape, many complex models can also be made.

Using Simscape, one can make various control systems and can test the performance of these systems in software without an application on hardware. If some error appears in the control system then it can be easily identified during the simulation. If this error containing a control system is applied to the physical model it can cause severe damages to the physical system. Therefore, Simscape is a very safe tool for simulation of various physical systems.

There are many physical domains present in Simscape like electrical, mechanical, thermal and hydraulic, etc. All the physical domains that are present in Simscape are shown in fig. 2.1.

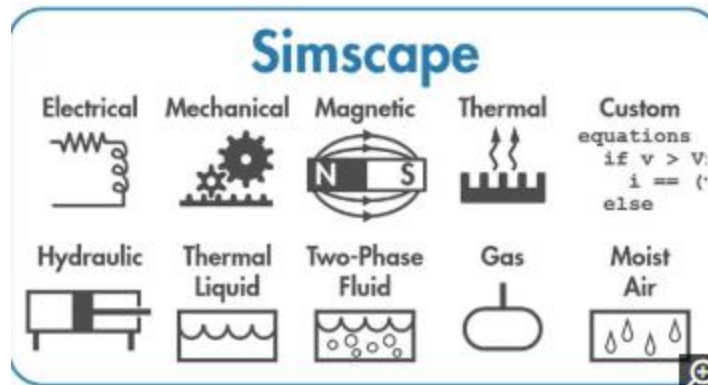


Figure 2.2 Physical domains in Simscape

There are thousands of components present in Simscape like springs, valves, resistors, and motors, etc. These components are connected to make a physical model. A simple mass-spring-damper system can be made by using mass, spring, and damper components present in Simscape. The complete model is shown in fig. 2.2.

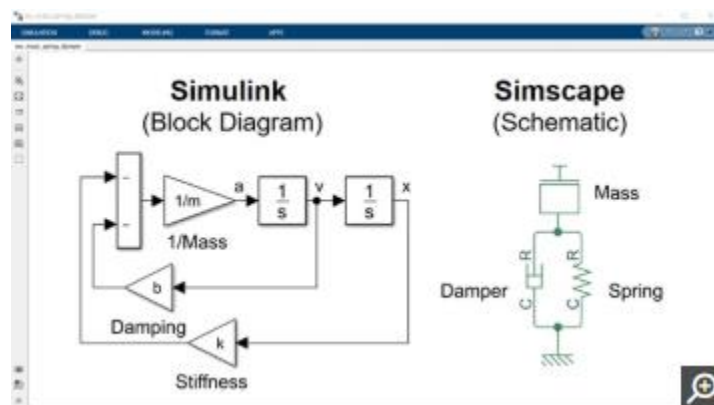


Figure 2.3 Mass spring damper system

Another multidomain model made using mechanical, fluid, and thermal components is shown in fig. 2.3.

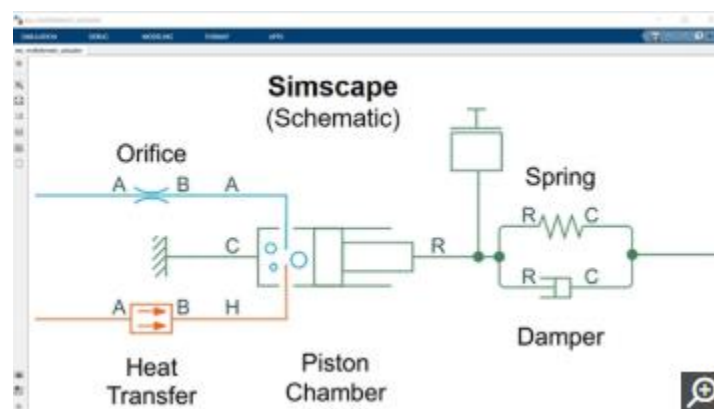


Figure 2.4 Schematic of multidomain model

CHAPTER 3 - COMPONENTS

The main components used in this project are:

3.1 SERPENT-1 ROBOT:

The Serpent robot is of SCARA (Selectivity Compliant Articulated Robotic Arm) configuration with its axis of movement in the X-Y plane. This robot finds its use in the industry mainly for pick and place. The Serpent has two DC motors to control the movement of its shoulder and arm in the horizontal axis and one for the rotation of the end effector. It also uses one hydraulic cylinder to control the movement of the end effector in the vertical position.

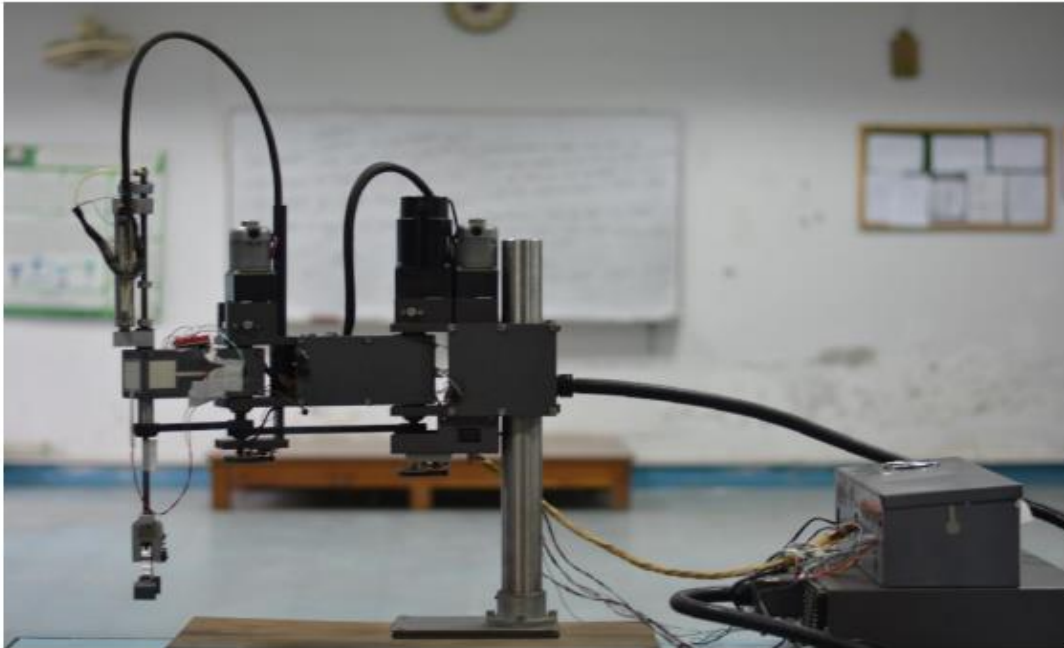


Figure 3.1. Serpent-1 Robot

The components attached with Serpent-1 Robot are as follow:

3.1.1 DC Motor:

Permanent magnet Direct Current motors are the most frequently used motors in DC applications. DC motor operates from a DC power source. By switching current between coils, the magnetic field's movement is achieved. This switching is done particularly within the motor itself. This switching action is called "commutation". Coils on the rotor are automatically commuted by the mechanical brushes. In a typical brushless motor, the permanent magnet electromagnetic field rotates. The rotating magnetic field interacts with several electromagnets commutated with electronics switches at the right speed and place.



Figure 3.2 DC Motor

Changing the polarity of operating power changes the rotation of the rotor. This is achieved using an H-bridge circuit. When compare to AC motors, PM motors have several advantages. They are usually physically miniature and lightweight. Additionally, there exists a linear relationship between speed and torque of a motor. For achieving high torque at low speeds, a permanent magnet can be considered. DC motors can operate from a battery, and their control units are simpler. They also have high efficiency. DC motors are widely used for a broad range of applications including robotics, power tools, and many automotive applications.

3.1.2 Motor Driver:

L298n is a very widely used and low-priced H-bridge motor driver. This driver has a vast application as it can drive two motors simultaneously. It can supply 2 amps per motor. The cooling portion is available in this drive as a huge flang is working like a sink and that drops the heat to zero. The disadvantage of using L298n is that it cannot be attached on the breadboard but the pins can be bent gently to attach its breadboard. Moreover, this problem can be resolved by using a breakout board and simply plugging the breakout board into the breadboard. The schematic in the following figure shows a Concise working diagram of the working of the L298n motor driver. There are three pins available for feeding the signal which include inputs, supply, and Enables controls which serve as the control of speed as well as for the control of direction. Many supply portion is also attached which make it a complete system.

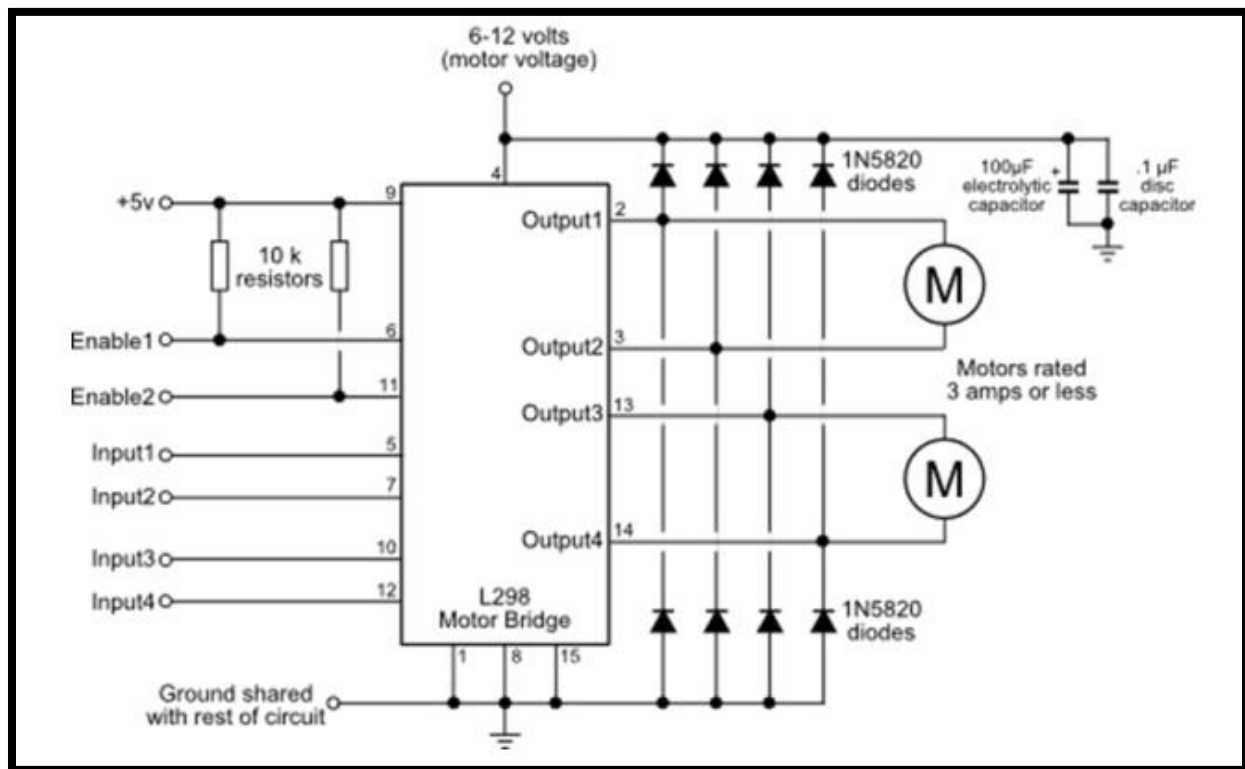


Figure 3.3. Circuit Diagram of L298

The direction of any motor can be controlled by applying the various transitions as described by the following table:

Input-1	Input-2	Actions
Low	Low	Motor breaks and stops
High	Low	Motor turns forward
Low	High	Motor turns backward
High	High	Motor breaks and stops

Figure 3.4. Direction control using L298

3.1.3 Encoders:

Shaft encoder is also known as a rotary encoder. It is a device that is used to convert shaft's angular position in corresponding Digital code. There are two types of rotary encoder: absolute and incremental. The absolute position of the shaft is given by the absolute encoder. Whereas the incremental encoder only gives changes concerning the current position and these can be used for position monitoring and also find velocity. Rotary encoders are mainly used when there is a need to find the precise position of the shaft. Few examples of this are industrial controls, robotics, photographic lenses, etc.

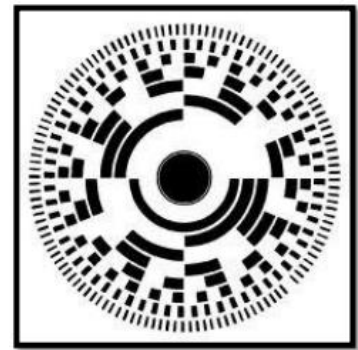


Figure 3.5. Absolute Encoders

3.1.4 Optical Absolute Encoders:

The disc of this optical encoder is made of plastic or glass material and the pattern which is inscribed over it is transparent. A source of light and a sensor to detect the light is used to read the pattern of the disc which is used to determine its position at any time. This pattern data can be processed by using a microcontroller to find the current position of the shaft. This encoder gives a specific dual analog code for every position. This code can be used to find the absolute angle of the shaft.

3.1.5 Incremental Encoders:

Incremental rotary encoders provide outputs in cyclical forms. Their resolution can be ten thousand counts per revolution or even more than that. The optical incremental encoder is widely used when a motor is of high speeds or higher precision is required to be achieved.



Figure 3.6. Incremental encoder

They normally have two channels with outputs called output-A & output-B, which are known as quadrature outputs, as both of them are ninety degrees out of

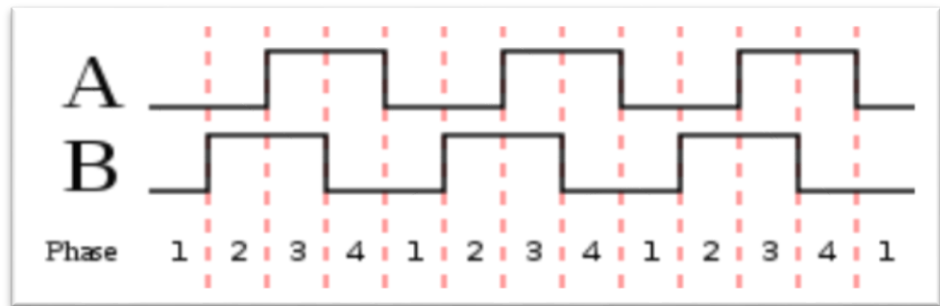


Figure. 3.7. Signal Diagram of Incremental Encoder

phase. These Digital waves are then forwarded to Counter which detects the transition of the signal and it serves as an accumulator. For reading the above values, both outputs are recorded through an intercept feature built in the respective program, and the above table is then utilized to decode the direction. As an example, if both of the output coming from the accumulator is tend to be zero then there is no change in overall position in any direction, and so on. A microprocessor is used to sample outputs of the encoder. The sampling rate needs to be high enough so that no change in state is missed. When encoder rotates fast, then the observer misses change in its output, so an invalid transition, such as 00 $\rightarrow$ 11, is detected, and the direction is undetermined.

In Serpent-1 Robot, incremental encoders are used to give positional feedback to Controller. While using such types of encoders it's impossible to locate the mean position. Mean Position is very important in our case as all movements on Cartesian coordinates are made using this position as a reference. So, for finding a mean position, limit switches are used.

3.1.6.Limit Switches:

A simple limit switch consists of an actuator that is mechanically linked to a set of electrical contacts. When an object causes the actuator to change state, the device makes or breaks the electrical connection.

In the Serpent robot, we have used two limit switches for locating the mean position of the robotic shoulder and elbow.



Figure. 3.8. Limit Switches

3.2 ARDUINO ATMEGA2560:

The Arduino-Mega-2560 is a microcontroller board based on specifications presented in the ATmega2560 datasheet. It has 54 digital I/O pins (15 pins can be used as Pulse Width Modulation output pins), 16 Analog pins, 4 hardware ports for serial communication, a sixteen MegaHertz crystal vibrator, one USB connection, one power jack, an ICSP module, and one onboard button for resetting the controller. It can easily be connected to a computer with a USB cable. The Mega 2560 is different from all previous board of this series because it does not use the FTDI Module. It uses ATmega16U2 (ATmega8U2 in the previous two boards of this series) which are programmed as a USB to Serial communication.



Figure. 3.9. Arduino Atmega2560

3.3 FORCE SENSITIVE RESISTOR:

A force sensor is such a resistor that is made up of such materials whose resistance varies whenever there is a force, pressure, or mechanical stress, applied to its surface. Most of the force-sensing resistors are made up of a conductive polymer, which alters its resistance in an almost linear fashion whenever force is applied to its surface.

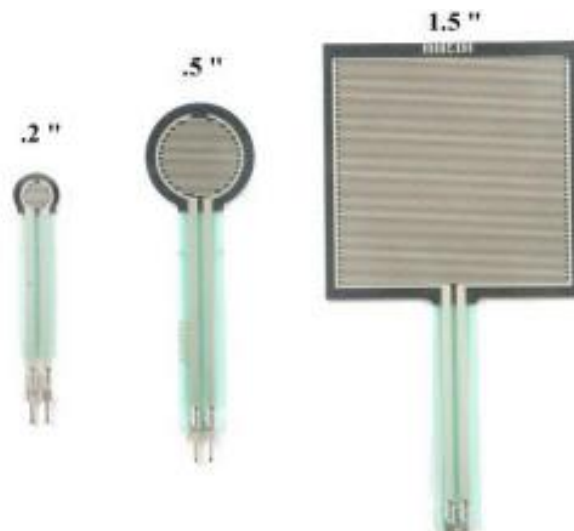


Figure. 3.10. Force Sensitive Resistors

We used three FSRs of range 1N to 20N on three sides of the end effector and made potential divider circuits to read the change in resistance of the force sensors using analog inputs of the Arduino mega 2560.

CHAPTER 4 - METHODOLOGY

In this project, two main things are focused on.

- Admittance Control
- Curve Fitting

4.1 ADMITTANCE CONTROL:

Human-Robot interaction is one of the advanced discussing topics for many years. Many industrial applications need interaction with the environment like polishing, deburring, machining, and many assembly tasks. The interaction of manipulators with the environment has many difficulties. The main difficulty is due to the contact's nonlinear behavior. There are many techniques used for controlling interaction with the environment and one technique is Admittance Control. In admittance control, user-applied force is taken as input while manipulator produces motion in the direction of the application of the force as an output.

The admittance law is:

$$F_{ext}(i) = M\Delta\ddot{x}(i) + B\Delta\dot{x}(i) + K\Delta x(i) \quad (4.1)$$

Here, “Fext” is the external force applied on the manipulator, “M” is the mass of manipulator, “B” is damping constant, and “K” is spring constant. Similarly, “Δx” changes in the position of end-effector, “Δ $\ddot{x}$ ” is the change in acceleration and “Δ $\dot{x}$ ” is the change in velocity of the manipulator. The iteration number is represented by “i”.

Damping constant and the spring constant is varied for controlling the interaction with the environment. A large value of spring constant means an increase in stiffness of the manipulator. The more the manipulator is stiff the more it will show resistance to motion produced by the application of force.

There are various methods for application of admittance control on the manipulator. The integration method is used for the application of admittance control on our manipulator [5]. Three simple steps are used for the application of this method on a manipulator. In the first step, damping constant B, spring constant K and external force Fext are taken as input and current change in acceleration (Δ $\ddot{x}$) is calculated as shown in equation 3.2.

$$\Delta\ddot{x}(i) = M^{-1}(F_{ext}(i) - B\Delta\dot{x}(i) - K\Delta x(i)) \quad (4.2)$$

The Fext is taken from the force sensors installed on the end-effector of manipulator. The three force sensors are installed. One sensor is attached in front of the end-effector while the other two sensors are installed on the left and right side of the end-effector. 20 values from sensors are read using analog pins of Arduino and then their average is taken. Then these values are calibrated and the forces are converted to three force vectors (F1, F2, and F3). The value read from the front sensor is taken as one vector (F2). While the value which is greater among right and left values are taken as another component (F1 or F3). Then, these vectors are added and a resultant vector is calculated.

The change in position (Δx) is obtained from encoder values. The encoder values are calculated every 110ms. For the first iteration, the previous change in acceleration is taken zero. Similarly, current and previous changes in velocities are also taken zero.

In the second step, for the calculation of the current change in velocity following equation is used:

$$\Delta\dot{x}(i) = \left(\frac{T_s}{2}\right) (\Delta\ddot{x}(i) + \Delta\ddot{x}(i-1)) + \Delta\dot{x}(i-1) \quad (4.3)$$

Here, “ T_s ” is the time taken for completion of one iteration. Similarly, “ $\Delta\ddot{x}(i-1)$ ” and “ $\Delta\dot{x}(i-1)$ ” are previous change in acceleration and previous change in velocity respectively. For the first iteration, both of these are taken zero.

In the third step, for calculation of the current change in position following equation is used:

$$\Delta x(i) = \left(\frac{T_s}{2}\right) (\Delta\dot{x}(i) + \Delta\dot{x}(i-1)) + \Delta x(i-1) \quad (4.4)$$

Here, “ $\Delta\dot{x}(i-1)$ ” and “ $\Delta x(i-1)$ ” are previous changes in velocity and previous change in position respectively. At the end of every loop, previous values are replaced by current values and current values are calculated in the next loop.

After calculation of position using admittance law, the actual position of the manipulator is calculated using the following equation:

$$X_{act}(i) = X_{ref} + \Delta x(i) \quad (4.5)$$

Here, “ X_{ref} ” is the desired position at which the manipulator should go. While “ X_{act} ” is the actual position of manipulator and “ Δx ” is the position calculated using admittance law.

After this, the PD controller is applied to the desired and actual angular position of the manipulator for producing torque that will be applied for moving the manipulator to the desired position. The linear position calculated using equation 3.5 and the position from encoders are converted to angular positions (q_d and q) using inverse kinematics and PD controller is applied on them for calculating target velocity using the following equation:

$$motor_{target_{vel}} = K_p(q_d - q) + K_d(\dot{q}) \quad (4.6)$$

Here, “ K_p ” and “ K_d ” are PD constants, and their values are adjusted for achieving the desired response (overshoot, settling time, etc.). While “ q_d ” is angular position calculated using admittance law, “ q ” is angular position obtained from encoders and “ $\dot{q}$ ” is the angular velocity of the manipulator. After this target velocity is calibrated for obtaining torque and then this torque is given to motors for producing motion.

In admittance control mode, there are three modes of operation. The first one is collaboration mode, in which the robot can be moved to any position by gently applying an external force on force-sensitive resistors installed on the end-effector and robot will move to that position without showing any resistance and stop as soon as the external force is removed and will stay there. The second and third modes are linked together. In teaching mode, the robot can be guided along any path by hand and this path will be saved in the form of x-y positions in an array in MATLAB workspace. And then in repeat mode, the robot will start to repeat the stored path in a loop and will move with the same speed at which it was moved originally. The first mode of operation is applied in this project where the robot will be moved by hand to draw a shape and the waypoints will be saved and these waypoints will be used to draw the intended shape.

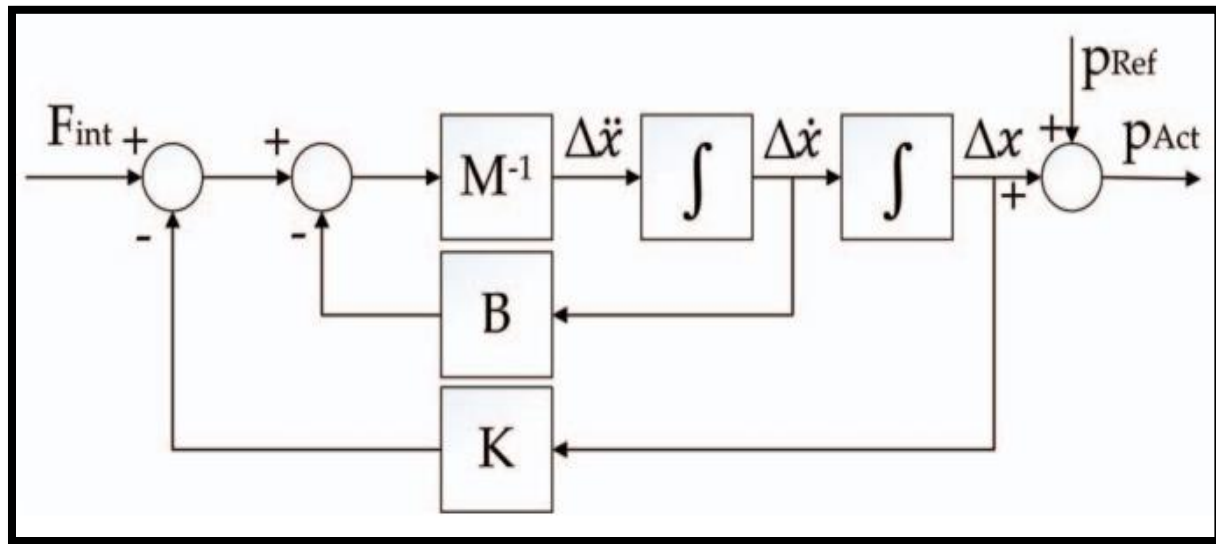


Figure 4.1. Block Diagram of Admittance Control

For taking the waypoints and then mimicking them, robotic kinematics will be taken into account. Kinematics concerns with the movement of robotic bodies. Kinematics is mainly divided into two other types:

- Forward Kinematics
- Inverse Kinematics

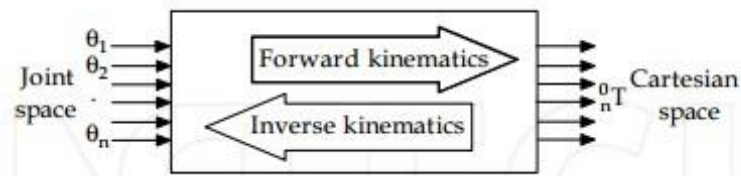


Figure 4.2. Forward and Inverse Kinematics

4.1.1. Forward Kinematics:

A robotic arm is composed of a series of links connected through revolute joints or prismatic joints to form a chain. Computing the parameters of end-effector using parameters of joint variables is termed as 'Forward Kinematics'. As our robot is a 2-link manipulator as shown in Fig 13.

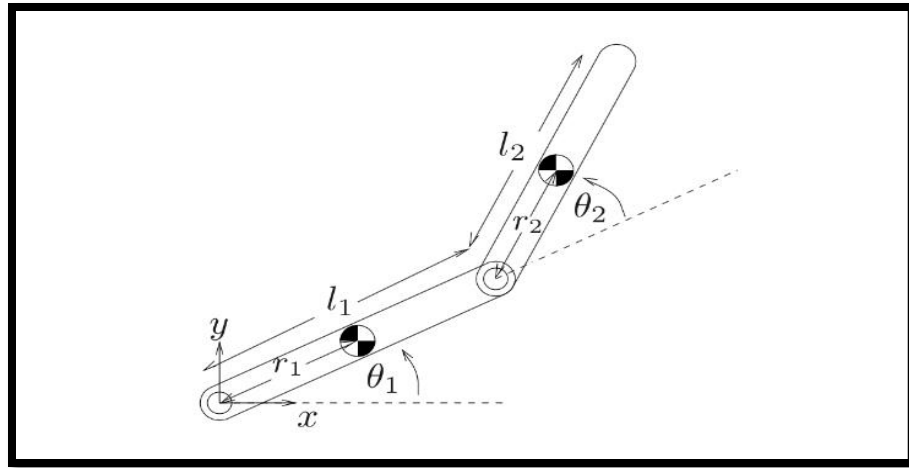


Figure 4.3. Two-Link Manipulator

Mathematical Equations of forward Kinematics for two-link manipulator are:

$$X = L1 * \cos(\theta1) + L2 * \cos(\theta1 + \theta2) \quad (4.7)$$

$$Y = L1 * \sin(\theta1) + L2 * \sin(\theta1 + \theta2) \quad (4.8)$$

4.1.2 Inverse Kinematics:

The computation of the position of robot joints as well as the orientation of joints given the position of the end-effector is called inverse kinematics. The inverse kinematics for the 2-link manipulator is:

$$\theta1 = \tan^{-1} Y/X - \tan^{-1} \frac{L2 \sin \theta2}{L1 + L2 \cos \theta2} \quad (4.9)$$

$$\theta2 = \cos^{-1} \left(\frac{x^2 + y^2 - L1^2 - L2^2}{2 L1 L2} \right) \quad (4.10)$$

4.2 CURVE FITTING:

The method of construction of curve or mathematical expression that efficiently fits different data points is known as curve fitting. The curve fitting process is either done through interpolation or smoothing. Where an exact fit to data is required, interpolation is used. Similarly, smoothing is used where the approximate fitting is required. There is various software in which the process of curve fitting is available like Maple, M-LAB, and MATLAB.

In our project, curve fitting played an important role in the intelligence part of our project. The intelligence part of our project included the learning and mimicking of shapes. The learning was done through admittance control while mimicking was done through curve fitting in MATLAB. When a shape was drawn using the admittance control technique, the position of end-effector in x and y coordinates was recorded in the form of data points. Then curve fitting technique was applied for fitting points according to a specific shape and new points were generated according to shape found through curve fitting. Then these newly generated points were fed to the manipulator using inverse kinematics and the desired shape was drawn. The curve fitting part was done using Simulink. The force was applied using force function for drawing the shape in the x and the y-axis. Then the position of end-effector was recorded and waypoints were fed to

curve fitting function where they were fitted according to shape and newly generated points were sent back to the manipulator for drawing of exact shape. Consider the example of the generation of a circle. The forces were applied in x and y direction and waypoints were taken and then curve fitting was applied on these points for drawing of a circle as shown in fig. 4.4.

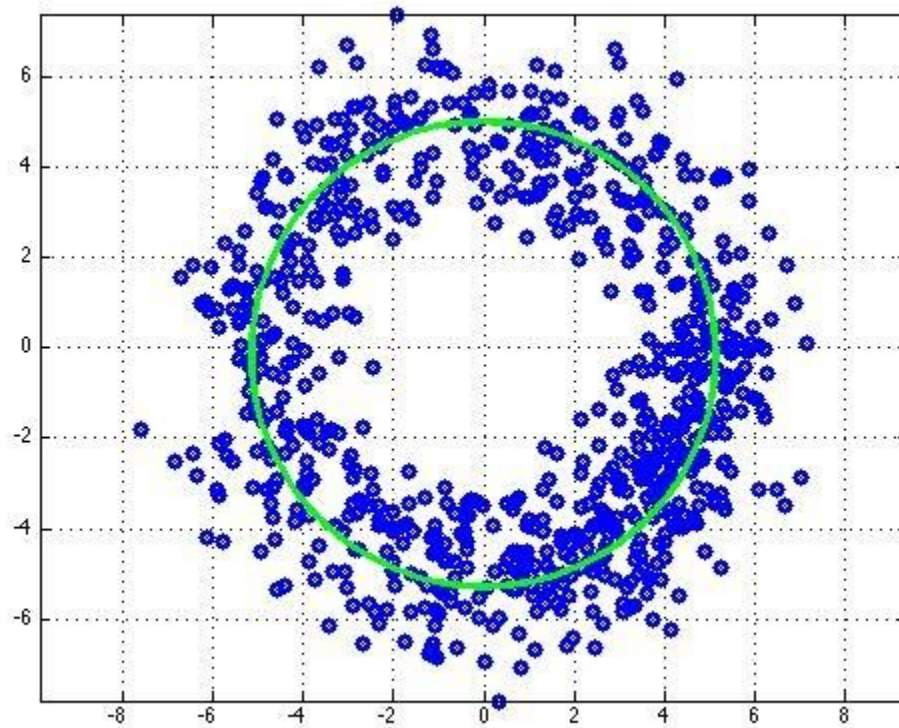


Figure 4.4. Curve fitting of a circle

CHAPTER 5:CONTROLLER PROGRAMMING

5.1.ARDUINO WORKING ENVIRONMENT:

The Arduino working environment contains different areas to facilitate code writing for the user. These areas are a message area, text editor (writing code), a toolbar with buttons (common function), a text console, and a series of menus. It works with the Arduino to communicate with it and also to upload the written code to Arduino. C-type language is used for writing code for



Figure 5.1. Arduino IDE

Arduino.

5.1.1.Sketch Making:

The codes written in the Arduino work environment are called sketches. Sketches in Arduino are inscribed in the text editor mode. The file extension of Arduino's written files is .ino. Whenever the code is saved or uploaded to Arduino the message area displays the warning and also gives the error which may be present in the code during development. The console gives the output in the text form from the Arduino. This also includes error messages and other useful information. The serial port and current board which is selected is also displayed in the lower right corner.

5.1.2.PIN Configuration:

The pins used of Arduino while programing the Serpent-1:

Table 1 Pin Configuration of Arduino

PIN	USE	PIN	USE
2	Enable pin of motor 2 (ENB)	30,31	Limit Switch Input
3,4	Input pin (IN3,IN4) motor 2	50	Encoder 2 power (LOW)
5,6	Input pin (IN1,IN2) motor 1	51	Encoder 2 power (HIGH)
7	Enable pin of motor 1 (ENB)	52	Encoder 1 power (HIGH)
22,23	Encoder Shoulder channel 1,2	53	Encoder 1 power (LOW)
24,25	Encoder Arm channel 1,2	A1	FSR 1 Input
A2	FSR 2 Input	A3	FSR 3 Input

5.2FUNCTION'S DESCRIPTION:

Many of the built-in functions used for making this project while many functions are made by himself for the smooth running of the project. It includes the function which derives the motor smoothly as well as the function responsible for getting information from the user in the form of Encoder and force Sensor. The given function that is made to reduce the complexity as well as for the ease of programming

5.2.1 Timer function:

This function is responsible for sampling the values of the force sensor as well as encoder values for further use. The built-in timer is used for calculating the time and by assigning the correct values in the time registers we get the sampled output from our device on time. This function is helpful in the smooth flow of the program.

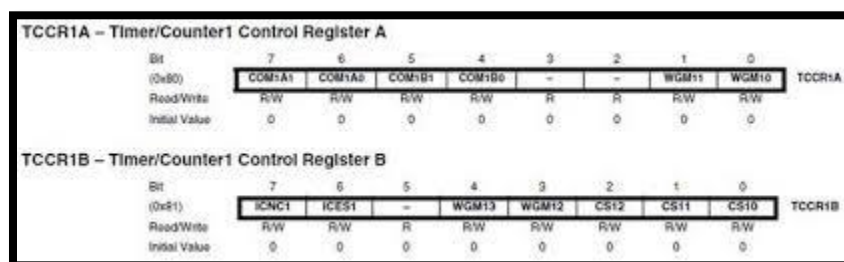


Figure 5.2 Arduino Timers Registers

5.2.2 Sampling input function:

Data is continuously coming from the sensor and the encoders while our system is somehow discrete which can analyze the input after a certain interval of time that's why after the sampling by using the timer function we take the moving average of that value and forward the adequate value towards the hardware input.

5.2.3 Actuation functions:

This function directly controls the PWM of the DC motor which in turn controls the speed of the motor. The PWM offset is provided in it which can be used for zero input. This offset is determined directly from the condition of gears. These offsets overcome the value of gear backlash and smooth flow.

To implement the desired position this function directly gives the input to the motor Drivers which is discussed in the component Section. The detailed working of how the position is extracted and how the manipulator achieves his goal is best described in the flow diagram below.

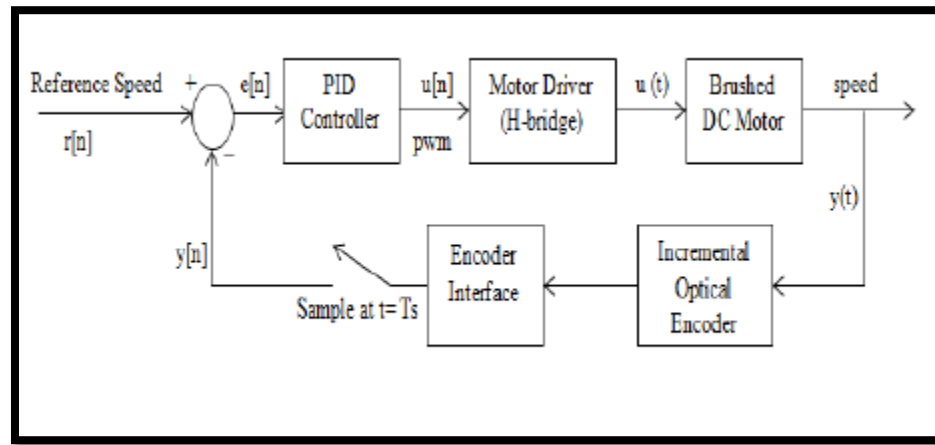


Figure 5.3 Motor Speed Control

5.2.4 Calculation functions:

For the computation of the admittance control in the project, there is a need of determining many variables which include the determination of forward and inverse kinematics as well as the desired and actual displacement computation to add the effect of stiffness constants and other material properties. This functions Compute not only the forward and inverse kinematics but also serves for the computation of jacobians to determine the velocities which are further processed in the admittance law.

5.2.5 linearization functions:

To protect the device from the damage the output is prevented from any sudden overshoot as well as to prevent any loss of sensor the input is passed through these functions which can linearize the value and regulate the output under safety condition which is allowed to send to the Cobot. This Serve as a regulator for the device while the limit in this function is designed according to the safety limits of the devices which include motor and motor drivers.

5.3 FLOW OF THE PROGRAM:

The given Arduino program Run in the following sections:

5.3.1: Acquiring the Position of the manipulator:

We are using the incremental encoder in this project which is used because of the low cost than absolute encoder. To overcome this situation two limit switches are added on the one end of the Robotic Arm. First the positive PWM moves the manipulator in an anticlockwise direction that after sometime detects the limit switches. As the limit switch output goes high The given location is assigned to the manipulator for further processing. This function thus eliminates the use of measuring absolute position by a respective encoder.

5.3.2: Acquiring Sensor values:

After a constant interval of time, the data is collected from the sensor which is used after the moving average. Position of the encoder is also fetched while checking that all the sensor and limit switches are working properly so that we will move to the processing phase in which by using displacement and force vector the desired outcome is processed

The force vector is computed by combining all the forces in the XY direction which is used for calculation in the admittance law.

5.3.3: Calculations:

Admittance law requires the computation of different varieties which is not directly given by the encoder values. It includes the further use of the kinematics equation for moving into the domain of Cartesian space and vice versa. While admittance Computation also requires the computation of the velocity and acceleration which is determined from the jacobian as well as from the integral method having algorithm discussed in the methodology section. The output of this phase is directly fed into the admittance law for calculating the desired displacement which includes the properties of stiffness and damping.

5.3.4: Application of admittance law:

After acquiring all the variables required for Admittance law the desired displacement is calculated from the application of admittance. It is considered the effect of stiffness and damping by using the displacement from the reference position. With the help of this displacement, it computes the velocity and acceleration which is calculated from the algorithms mentioned in the literature. The value computed is in Cartesian space it is further moved to angular space for further processing.

5.3.5: Calculating PWM:

The output from the admittance is in the form of Cartesian space while the input to the motor can be related with the angular motion, therefore, an inverse kinematics block is first used to convert it into the angular space while the angle required is mapped into the PWM which indirectly measure the direction as well as the speed of the manipulator.

5.3.6: Linearizing the output:

The given output is not directly applied to the motors of the manipulator because it might include the overshoot values which not only damage the circuit but also damage the motor through which we can move to further destruction. To avoid this we will limit the output value so that it is constrained in a limited range. All the data in the given range is safe to handle and this range is selected considering the safety limits of our hardware.

5.3.7 Position Control:

The position corresponding to force is not achieved in a single iteration rather it is achieved by multiple iterations as long as the displacement values converge toward the required value. This value is computed by using PID control which is performed like in the flow diagram given below.

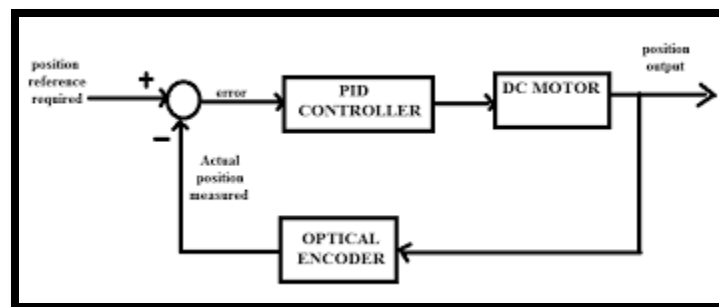


Figure 5.4 PID control for position control

CHAPTER 6- SIMULATION

Before moving into the hardware work simulation is done on Simscape where admittance control followed by a position control is applied. A given trajectory of a circle is provided for a given time frame. The given trajectory is passed through the admittance law where the desired position under the input of damping and stiffness is created which is further given to the dynamics of the manipulator. All the physical coefficient is in the Robot Dynamics model. The position and velocity for the physical model are then generated from the Robot dynamics. The code present in the Robot Dynamics function is available in appendices. The Simscape model thus uses the position and velocity to generate a physical model. Angular position and angular velocity are provided to the revolute joint which is used to generate a physical display. All the scopes are arranged in such a way that it will display the commanded and desired position. As the forces on the end of 2 link manipulator are applied it moves in the respective axis. The commanded position will follow the desired position.

For simulation, the Simscape tool is used. A 2 DOF manipulator is designed in Simscape using revolute joints as shown in fig. 6.1. It is the same as our hardware setup.

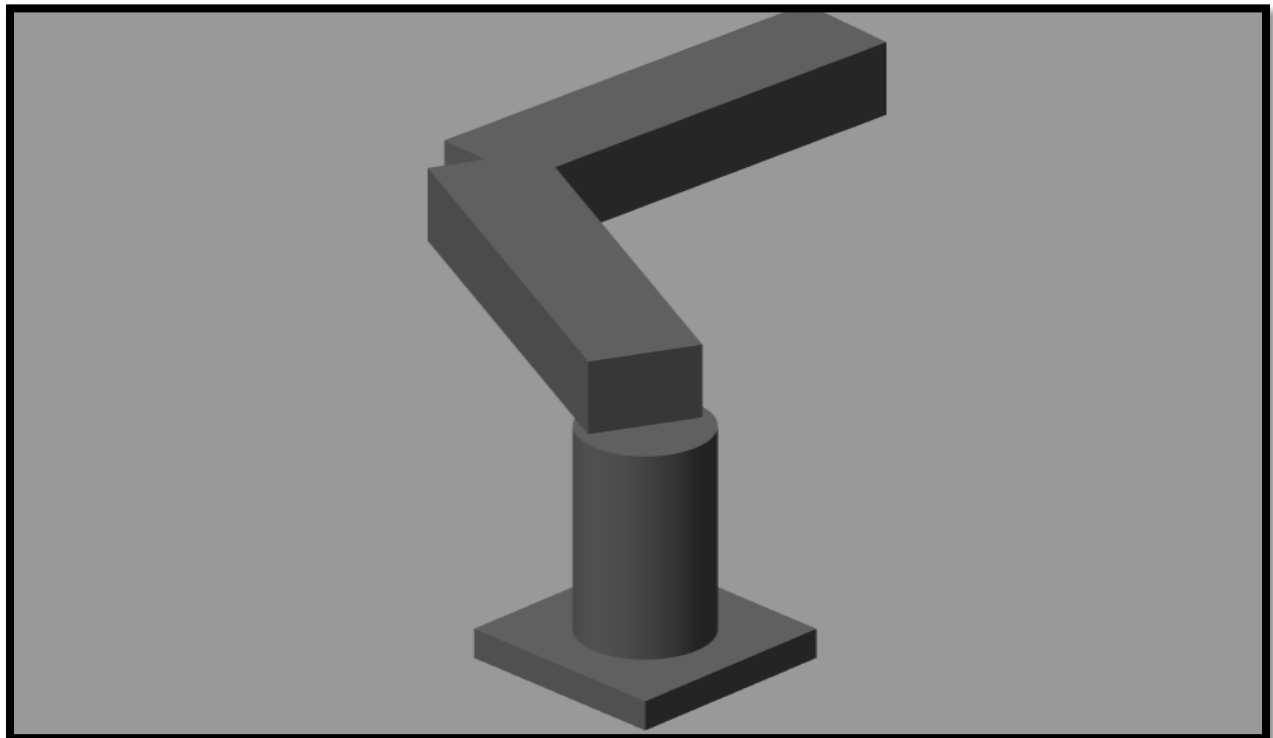


Figure 6.1. Two DOF Manipulator in Simscape

The internal model of two DOF manipulator in Simscape is shown in fig. 6.2. This internal model is generated using components present in the mechanical domain of Simscape. The base of the robot is shown on the left-most side of the figure. Two revolute joints can also be seen in the figure. Forward and inverse kinematics are applied to these two revolute joints. These forward and inverse kinematics tells about the position and angle of these revolute joints. Two external forces are applied to the end-effector and it can be seen in fig. 6.2.

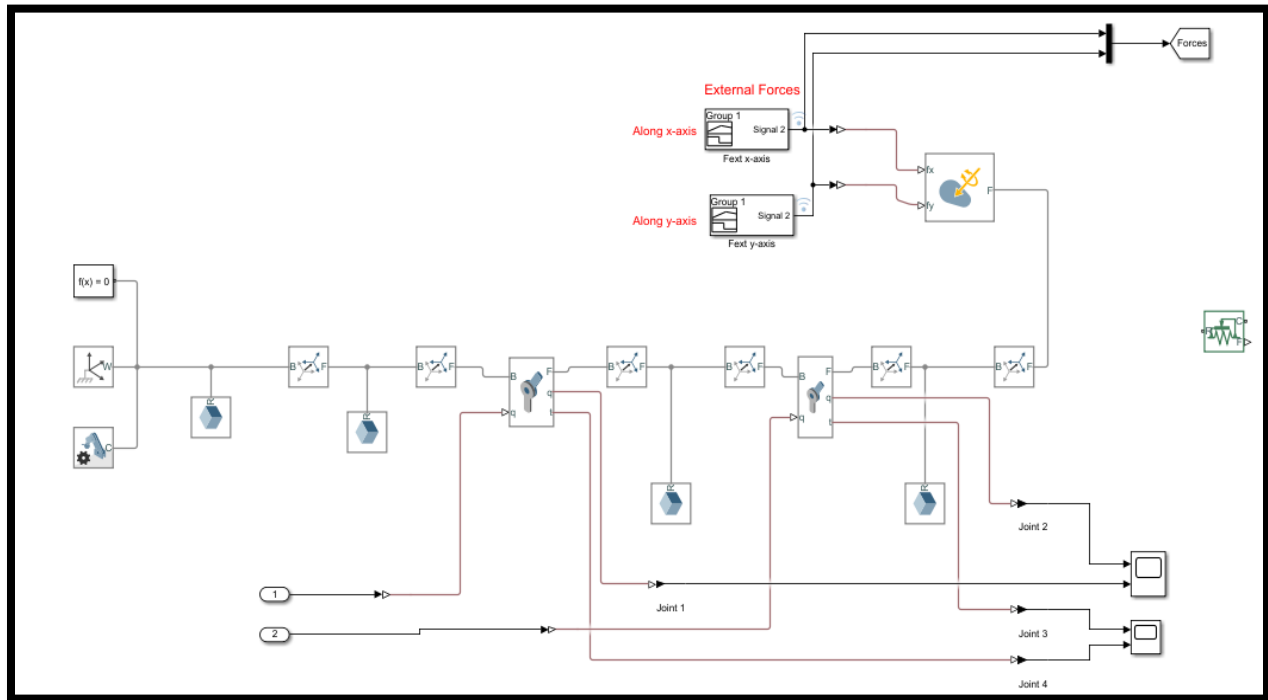


Figure 6.2. Simscape Model of Robot

The dynamics of the robot are shown in fig. 6.3. In fig. 6.3. there is a robot dynamic function present and this function is programmed in MATLAB. The inputs to this function are external forces, actual linear position, actual angular position, desired linear position, linear velocity, and angular velocity. While the output of this function is the angular acceleration of the manipulator. Then double integration is applied to this output for getting angular velocities and angular positions of both revolute joints. Then these angular positions are fed to revolute joints and forward kinematics is applied for getting linear positions of these joints which are used in further process.

The Simulink model of admittance law is shown in fig. 6.4. It can be seen that external force is provided as input while values of mass, spring constant, and damping constant are provided through fixed gain. The output of this admittance law is the desired position and current position and velocity of the manipulator. It can be seen that admittance law is a close loop system as there are two feedbacks present.

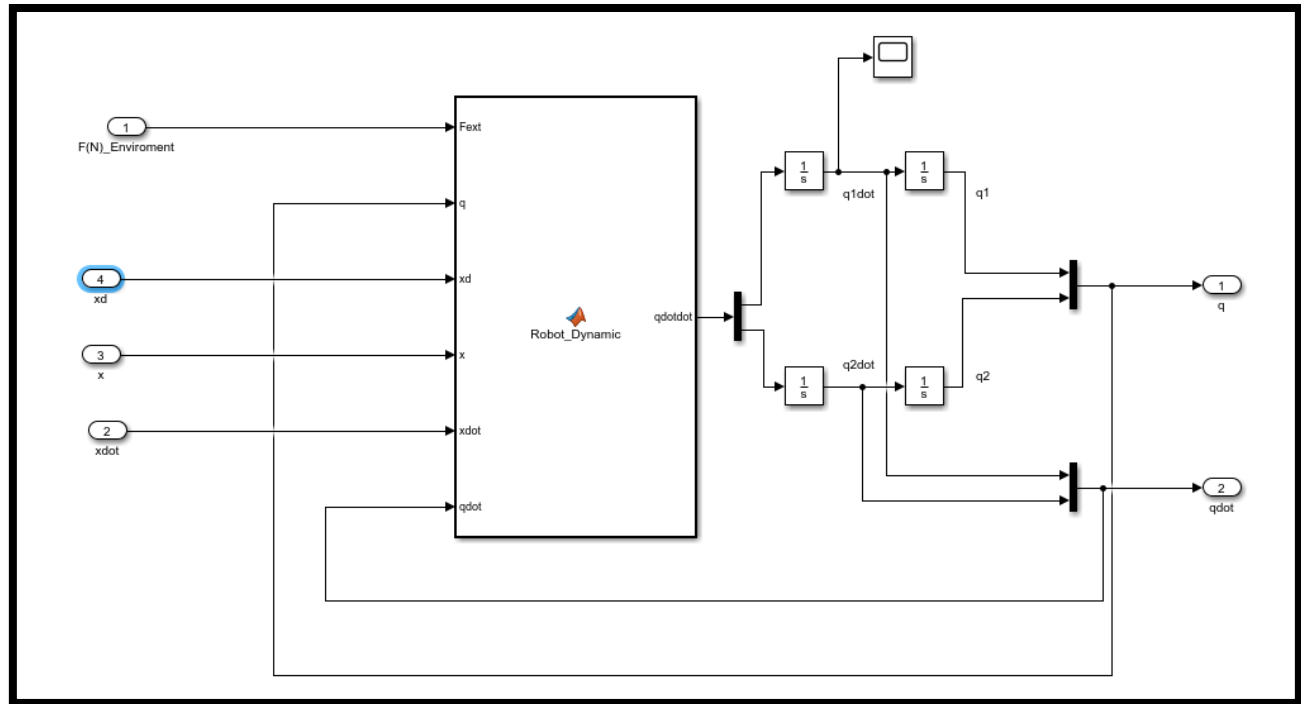


Figure 6.3. Robot Dynamics in Simulink

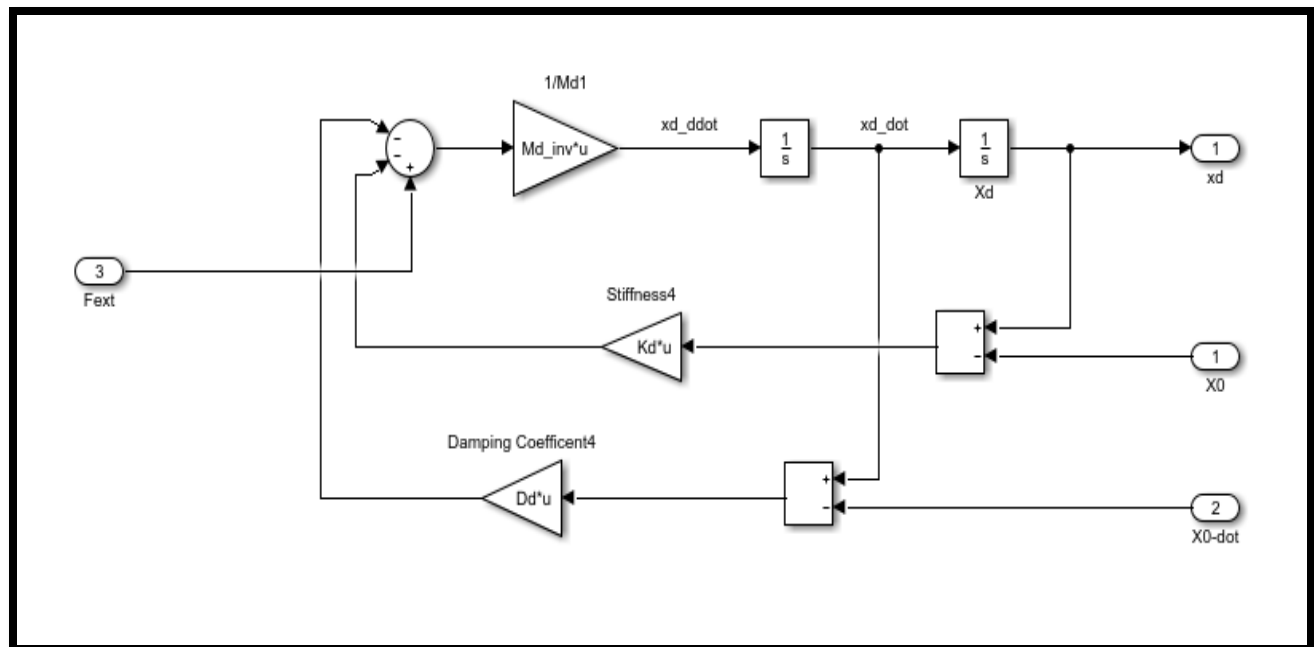


Figure 6.4. Admittance LAW

After the application of admittance law, curve fitting was applied to the manipulator. For it, the function block of curve fitting was added in the Simulink model. The input to this curve fitting function is waypoints generated when admittance is applied and robot moves to draw the shape. The output of the function is waypoints that are obtained using curve fitting. These waypoints are

fed to the desired trajectory as shown in fig. 6.5. Then the robot moves according to this desired trajectory and draws the intended shape. The complete Simulink model is shown in fig. 6.5.

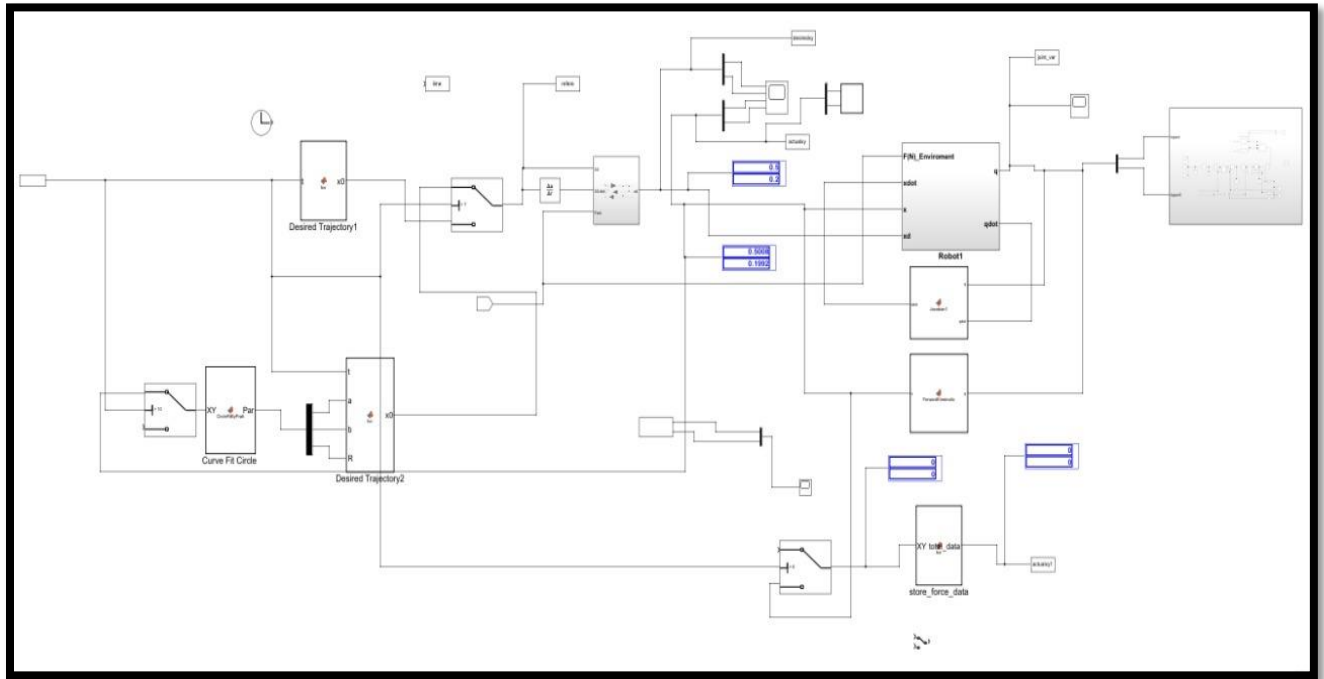


Figure 6.5. Complete Simulink Model for Admittance Control

CHAPTER 7- RESULTS

7.1 SIMULATION RESULTS:

In admittance control, force is applied at end-effector in direction of desired motion and manipulator moves in that direction. The same process is done in Simulink in MATLAB. In the simulation, force is applied in both x and y-axis and the manipulator follows the desired path as shown in fig. 7.1. Force in the x-direction is applied from 1-3 seconds and during this robot's desired position is changed and then it follows then desired path. When the force is removed the desired position comes back to its initial position and robot follows it. Similarly, force in the y-direction is applied from 4-6 seconds, and for this case also desired path changes and the robot follows it and when force is removed it comes back to its initial position.

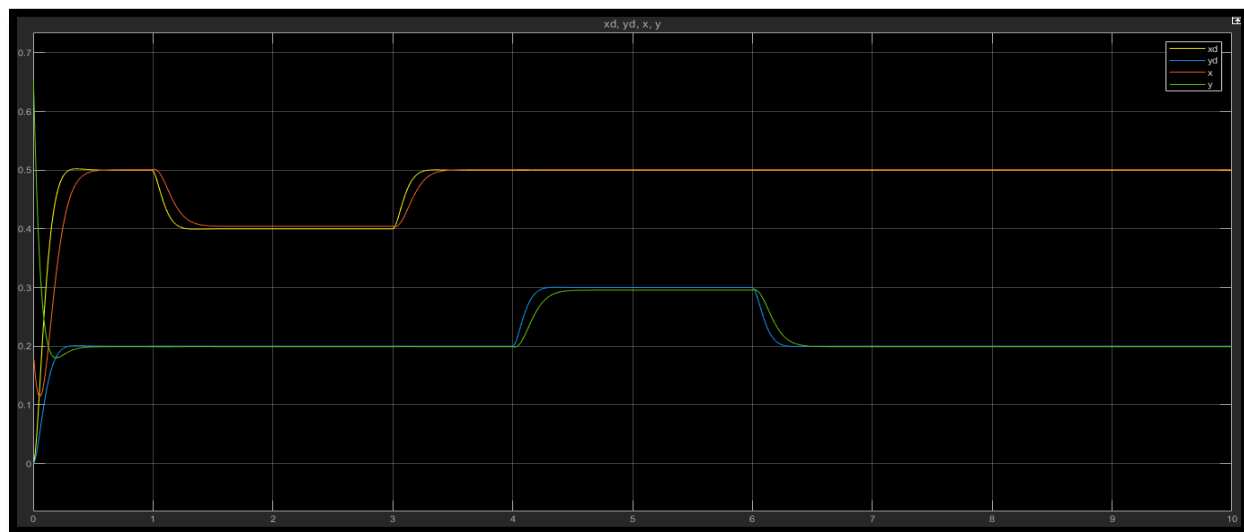


Figure 7.1. Simulation Result of Admittance Control

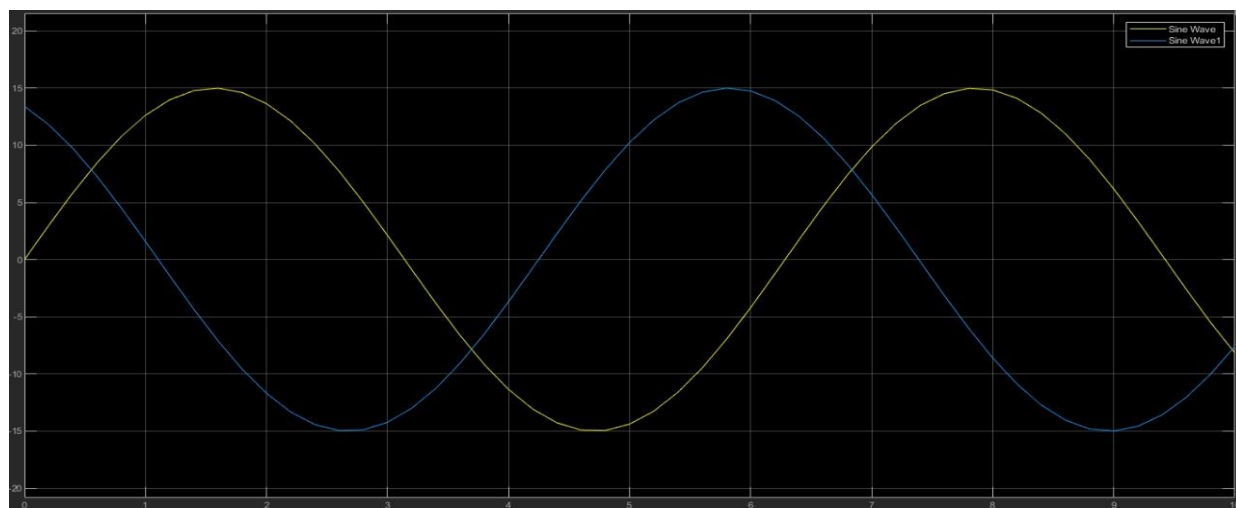


Figure 7.2 Input For the drawing of Circle

The circle was drawn in Simulink using admittance control. The forces applied in x and y direction for the drawing of the circle are shown in fig. 7.2.

The waypoints for the circle created by the application of the above points were recorded and fed into the curve fitting function block present in the Simulink model. Then curve fitting was applied on these points for the generation of a circle with the desired radius. The circle drawn by manipulator with the application of force and circle created using curve fitting is shown in fig. 7.3. In fig. 7.3. the blue color circle is the circle drawn by manipulator using admittance control and the red color circle is obtained using curve fitting.

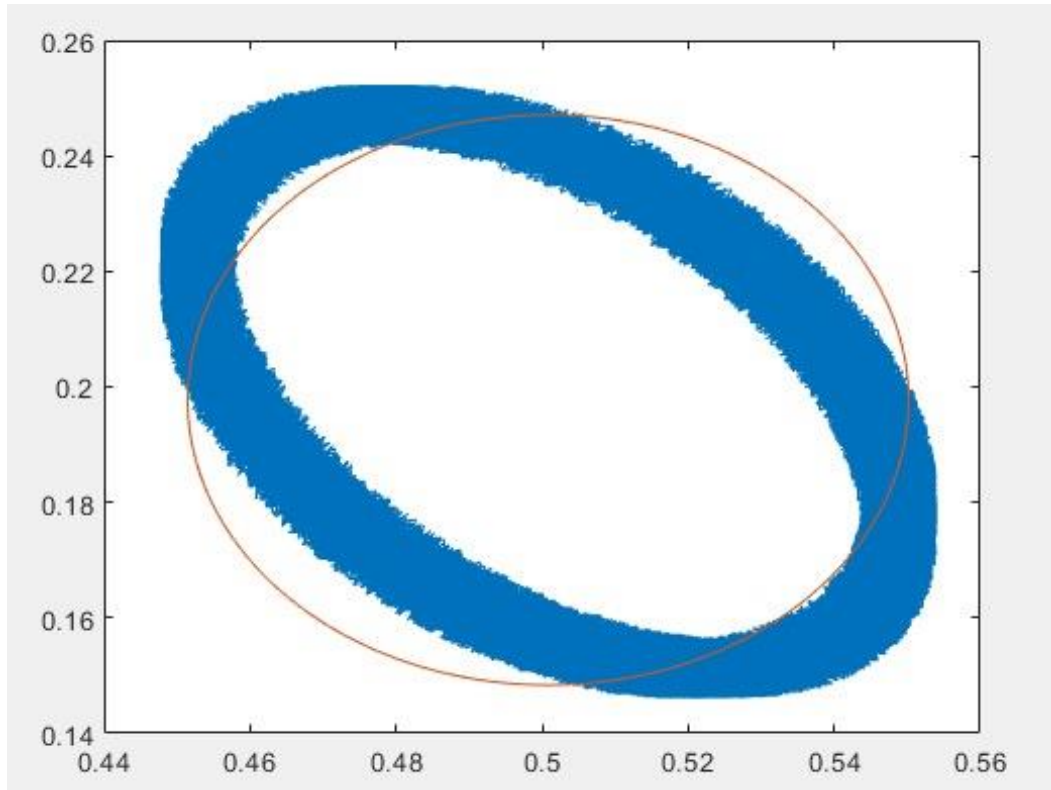


Figure 7.3. Circles drawn using Admittance control and curve fitting

The above figure is for a high amount of data and for a given value of noise which is a function of a normal random variable.

For the smaller amount of data, it looks like the figure given below

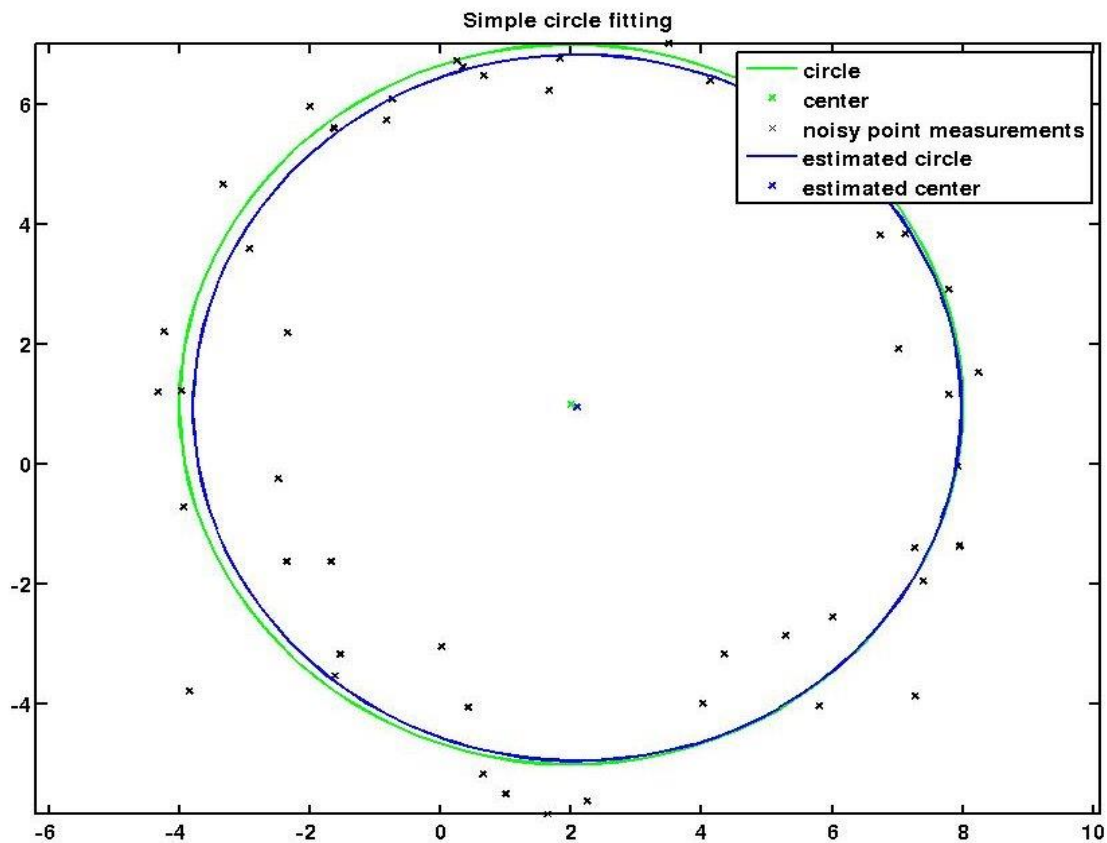
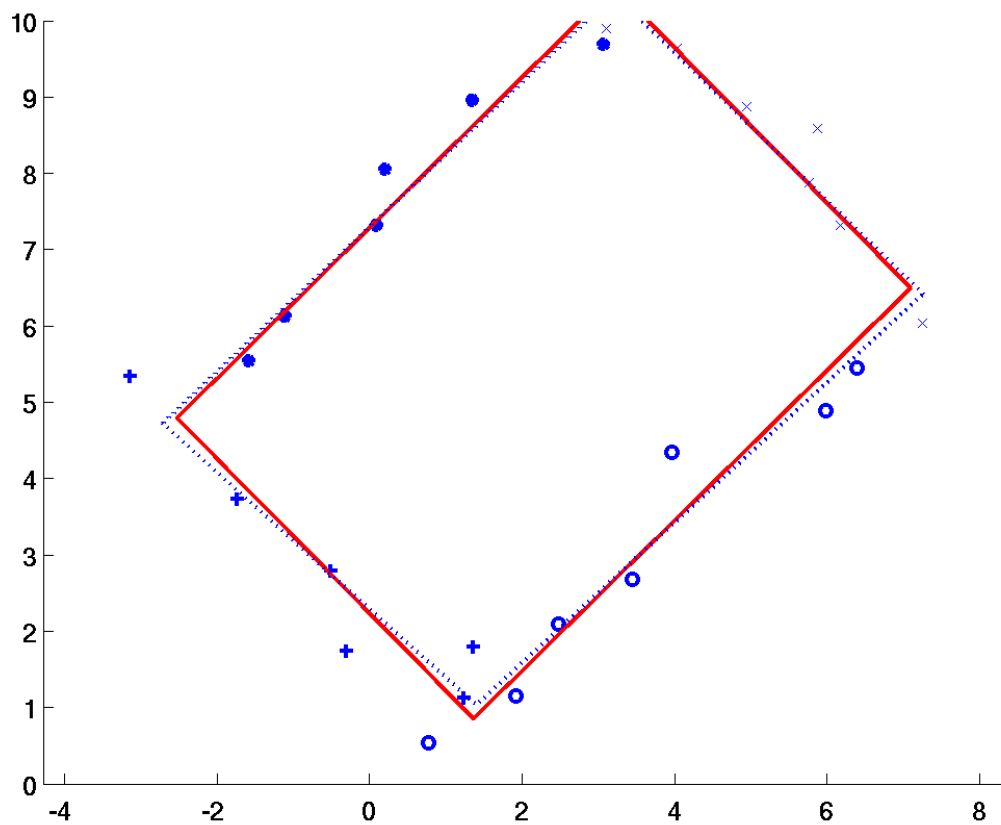


Figure 7.4 Circle Fit

The data point taken from the manipulator during its motion from the user gives the noisy data as the data is passed through the curve fit it will give you the location and radius of that fitted Circle

For rectangular Fit :*Figure 3 Rectangle Fit*

So that the user wants to draw a rectangle the rectangle is fitted on it and it gives the equation of diagonal through which we compute the rectangle.

7.2 HARDWARE RESULTS:

Hardware Result is computed when we are using the Arduino software with the physical system. The given picture is taken from the working hardware

7.2.1 Application of force:



Figure 7.6 Force on End effector

7.2.2 Displacement by force:



Figure 7.7 Displacement corresponds to force

7.2.3 Return to reference position:



Figure 7.8 Return to reference position

CHAPTER 8- CONCLUSION

8.1.FUTURE ENHANCEMENTS:

There is a scope of a lot of improvement in this project. Some of them are discussed below.

8.1.1.Better Sensors:

The Project performance can be improved using torque sensors instead of using force sensitive resistors (FSRs) as resolution and accuracy of FSRs is not good. Another advantage of using torque sensors is that they can detect abnormal forced being applied to ant part of the robot's body. Current sensors with better resolution and less noise can also be used for better results. The current sensor ACS 720 works on the Hall Effect principle so its output values get effected by magnetic interference hence gives noisy values.

8.1.2.Use of Harmonic Gear Drives:

Harmonic drives provide a large gear ratio with only a small increase in the overall weight of the system. So, they can be used instead of normal gear trains, which are bulky and heavy. Another advantage of the harmonic gear drive is that they have zero backlashes, which would improve the overall response and performance of the robot.

8.1.3.Use of Image Processing:

Instead of the application of force for drawing a particular shape, the image processing technique can be used for checking which shape the user wants to draw and then using this technique, making the robot able to draw a shape which is shown in the image. It is an advanced technique and can be implemented using knowledge of Machine Vision.

8.2. ENVIRONMENT AND SUSTAINABILITY GOALS:

It was a challenge to apply the admittance control scheme on the manipulator and draw a shape and then make the robot able to draw that intended shape by itself. The techniques that have been used in various industries and researches for applications of admittance control in robots are utilized in this project. This project aims to make the robot able to collaborate with humans to interface a very basic level in intelligence in robotic systems. The collaboration is achieved with the application of the admittance control scheme on the 2DOF manipulator and basic level intelligence is achieved with the help of a curve fitting technique.

The robot is made able to collaborate with humans using the admittance control scheme. The robot can draw shapes with the application of force on force-sensitive resistors and move it in the desired direction. Then with the application of curve fitting, the robot can draw the exact intended shape. Our work deals with the control of the mechanical interaction of robots with different objects in industries. Similarly, it is also suitable for ease of coding. In industry, if someone wants to do multiple tasks with it, he/she does not need to change the program every time. He/she just has to move the robot on the desired trajectory with the application of force on force sensors and the robot will learn it and then will start following it.

8.3.CONCLUSION:

We successfully applied the admittance control scheme on Serpent-I Robot. Using this control scheme made the robot able to draw the shape. After drawing some random shape, we implemented a curve fitting process on this shape for converting to an exact shape. Then we made the robot able to draw this exact shape.

CHAPTER 9-APPLICATIONS

There are numerous applications of the Impedance Control Scheme

9.1.PATH GUIDING:

When robots are used in the industry to perform an assigned task, it requires the need to program the robot for that specific task. Experts are required to program the robot and make it work whenever there is any change in the environment or there is a need to modify the process. Hence, the robot needs to be reprogrammed and again the expertise of the experts is required. This is a very time-consuming process and costly as well.

One of the major applications of the Impedance Control is the path planning exercise. While the robot is working in the impedance control mode, it allows human interaction. The human worker can change the mode to the learning mode and can move the robot to any desired location and store those points until the task is performed and then change the mode of operation from learning to work. The robot will then start performing the specified task using those stored points. This saves the time and cost which is required to program the robot for the same task and owing to this, workers can easily perform path planning exercises without any programming skills.

9.2.SAFETY FACTOR:

Today robots are an essential part of the industry for various purposes. The main approaches involving the Control of these robots revolve around position control. The major flaw of this control scheme is that it does not account for human intervention. If a human worker is present in its workspace it won't stop working and might injure the worker this is why there caged to prevent humans injure a red line is a place around the robots to let others know that beyond this point their safety is not grunted.

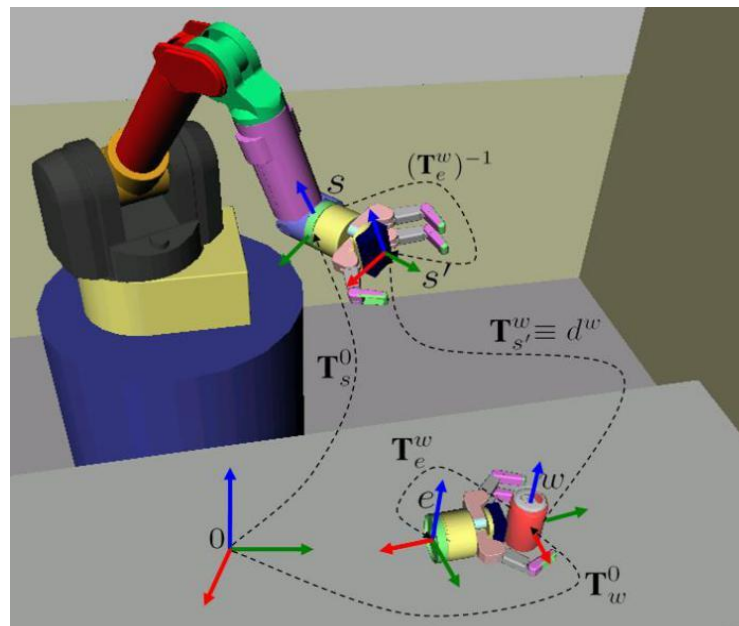


Figure 8.1. Path Guiding

If there is a need for human intervention for incase of maintenance need or quality check then that will shut down and the desired task is performed but with impedance control scheme implemented it allows human intervention. As it detects the unusual force acting on it will stop working until these forces are present when a human operator removes these forces it will resume its working form where it left.

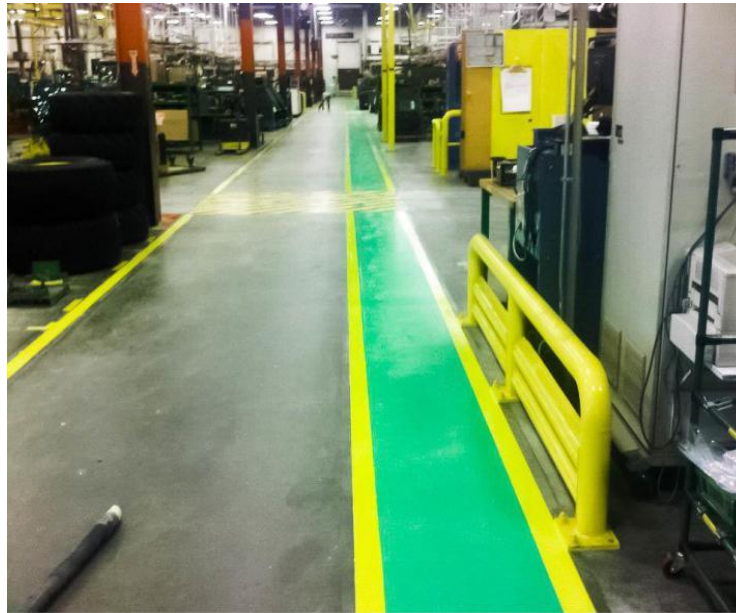


Figure 8.2. Safety Lines

9.3.AUTOMATED ASSEMBLY:

Normally in industries, assembly operations are performed manually. They can be done either at assembly workstations or on assembly lines with multiple stations. The recent advancements in robotics and automation have led to automated assembly lines being used as shown in fig. 23. Incorporating collaborative control in the machines operating on assembly lines has many advantages. Human operators can safely work nearby of these robots. Because any contact between humans and robots will not harm anyone. Both robots and operators can assist each other in their tasks and work as a team.

9.4.AUTOMATED PRODUCTION LINE:

The automated assembly line normally consists of several workstations that are connected by a transfer system usually moving belts to move parts between the two or more than two stations. These lines are normally set up to achieve the production of large quantities. The production size can be of millions of product units, and it is designed to work for several years. Each of the stations is designated to perform its designated task. The part which passes through all of the work stations is constructed in several steps as it moves along the assembly line. An unfinished part enters from one side of the line, passes through all of the workstations, and a finished part is received at the other end of the last work station.



Figure 8.3. Automatic Assembly Line

While the machines are working normally there is one part at each work station that is being processed. This is done so that many parts are being operated on by the machine at the same time and with each cycle passing we get a newly finished part. All the task which are being performed to produce finish well must be in synchronization to there is no problems in end. More advanced automated lines are being controlled with the help of PLCs. Which are specially designed computers to help form links between industrial equipment and can perform the kinds of operation which require time and sequence for their operation.



Figure 8.4. Automatic Production Line

REFERENCES

- [1] John J. Craig, Introduction to Robotics, 3rd ed. Harlow: Prentice-Hall, 2005.
- [2] P. Song, Y. Yu, and X. Zhang, "Impedance Control of Robots: An Overview," 2nd International Conference on Cybernetics, Robotics and Control, 2017.
- [3] N. Hogan, "Impedance Control: An approach to manipulation," Proc. American Control Conference, IEEE, 1984, pp. 304-313
- [4] N. Hogan, "Impedance and Admittance control of industrial robots," Robotics and Computer-Integrated Manufacturing, vol.1, 1984, pp. 97-113.
- [5] Gustavo J. G. Lahr, Joao V. R. Soares, Henrique B. Garcia, Adriano A. G. Siqueira and Glauco A. P. Caurin, "Understanding the Implementation of Impedance Control in Industrial Robots," XIII Latin American Robotics Symposium and IV Brazilian Robotics Symposium, 2016, pp. 269-271.

APPENDIX

Arduino Code:

```
#include "math.h"

//Some global constants

const float pi = 3.14159265358979323846;

//structures

struct store_2_double          //datatype for storing 2 floating values
{
    double val1;
    double val2;
};

//Functions

void setup_motors();

double get_value_FSR_1(); //To get the values of FSR-1
double get_value_FSR_2(); //To get the values of FSR-2
double get_value_FSR_3(); //To get the values of FSR-3

void motor_actuate(int, int); //actuates the motor according to input parameters given

store_2_double understand_commands(); //gets input string and executes it

float string_to_float(String);    //converts string to float

double motor_1_vel_2_pwm(double);
double motor_2_vel_2_pwm(double);
double motor_1_vel_to_pwm(double);
double deg2rad(double);
double rad2deg(double);
double sind(double);
double cosd(double);

store_2_double forward_kine(double, double);
store_2_double inverse_kine(double, double);
store_2_double jacobian(double, double, double, double);
```



```
store_2_double inv_jacobian(double, double, double, double);
store_2_double jacobian_transpose(double, double, double, double);
void setup_sample_timer();           //setups timer 5 for use in reading encoders
void encoder_read(boolean, boolean, byte); //reads encoder position

//driver pins
int EnB = 2;
int In3 = 3;
int In4 = 4;
int In1 = 5;
int In2 = 6;
int EnA = 7;

//force sensor stuff
const int samplesize_forcesensor_1 = 5;
const int samplesize_forcesensor_2 = 5;
const int samplesize_forcesensor_3 = 5;

//encoder stuff
volatile int encoder_pos[2] = {0}; // to save count values for 2 encoders
volatile int encoder_vel[2] = {0}; // to save current values of 2 velocities measured from encoders
volatile int encoder_acc[2] = {0};
volatile bool allow_sampling_input = 0;
volatile bool allow_sampling_commands = 0;
volatile bool allow_velocity_transition = 0;
volatile bool allow_printing_curr_pos = 0;

//main loop stuff
bool limit_flag[2] = {0, 0};
double local_cart_target[2] = {0, 0.39};
double curr_cart_position[2] = {0, 0}; //current position in cartesian coordinates
double curr_cart_velocity[2] = {0, 0}; //current velocity in cartesian coordinates
double curr_cart_acceleration[2] = {0, 0}; //current acceleration in cartesian coordinates
```

```
double curr_encoder_pos[2] = {0, 0};    //to save current encoder pos
double curr_motor_pos_deg[2] = {0, 0}; //to save current motor position in degrees
double curr_encoder_vel[2] = {0, 0};    //save current motor velocities, ticks per sec
double curr_motor_vel_deg[2] = {0, 0}; //save current motor velocities, deg per sec
double curr_encoder_acc[2] = {0, 0};    //save current motor velocities, ticks per sec
double curr_motor_acc_deg[2] = {0, 0}; //save current motor velocities, deg per sec
double curr_motor_vel_deg_from_jacobian[2] = {0, 0}; //to test jacobian and jacobian inverse
double motor_target_vel[2] = {0, 0};
//current direction determining stuff
bool curr_neg_dir[2] = {0, 0};
store_2_double temp_store_2_double;
double current_1 = 0;
double current_2 = 0;
double force_1 = 0; //for force_1
double force_2 = 0; //for force_2
double force_3 = 0; //for force_3
double theta_force_1 = 0; // Theta of FSR-1 in local coordinate system
double theta_force_2 = 0; // Theta of FSR-2 in local coordinate system
double theta_force_3 = 0; // Theta of FSR-3 in local coordinate system
double vector_force_1[2] = {0, 0};
double vector_force_2[2] = {0, 0};
double vector_force_3[2] = {0, 0};
double pwm_offset_m1 = 38; // To be added in calcualted pwm to have some motion (To have some motion at min pwm)
double pwm_offset_m2 = 50; // To be added in calcualted pwm to have some motion (To have some motion at min pwm)
// Impedance Control
const double Mm[2][2] = {{0.5, 0}, {0, 0.5}};
const double m1 = 2;          //kg
```

```
const double m2 = 0.85;          //kg
const double L1 = 2.5;          //m
const double L2 = 1.5;          //m
const double Lc1 = L1 / 2;      //m
const double Lc2 = L2 / 2;      //m
//const double g = 9.8;          //m/s^2
const double g = 0;             //m/s^2
const double I1 = 0.06125; //kg m^2
const double I2 = 0.02042; //kg m^2

//double input_Km[2] = {100, 100};
//double input_Bm[2] = {3, 3};
double Km[2][2] = {{10,0},{0,10}};
double Bm[2][2] = {{4,0},{0,4}};
double conver=12/1024*10^-6;
unsigned long last_time_sampled=0;
unsigned long now;
int operation_mode = 3;

double q[2]; //to save current motor position in degrees
double qd[2];
double q_dot[2]; //to save current motor velocity in degrees
double p1, p2, p3, p4, p5;
double M[2][2];
double C[2][2];
double G[2];
double J[2][2];
double MJ;
double IJ[2][2];
double IJT[2][2];
```

```
double Mx[2][2];
double I[2][2] = {{1, 0}, {0, 1}};
double IMm[2][2];
double IKm[2][2];
double MMm = 0.25;
double Kmm=10000;
double Kp[2]={2,3.5};
double Kd[2]={0.12,0.12};
double Jdot[2][2];
double JT[2][2];
double a[2];
double a1[2];
double a2[2];
double a3[2];
double F[2];
double Inv[2];
double xd[2]={0,0};
double xd_dot[2]={0,0};
double xd_dotdot[2]={0,0};
double prev_xd[2]={0,0};
double prev_xd_dot[2]={0,0};
double prev_xd_dotdot[2]={0,0};
double ref_x[2];
double actual_x[2];
double delta_x[2]={0,0};
double delta_x_dot[2]={0,0};
double delta_x_dotdot[2]={0,0};
double prev_delta_x[2]={0,0};
double prev_delta_x_dot[2]={0,0};
```

```
double prev_delta_x_dotdot[2]={0,0};

double Ts;

double Inv1[2][2];

double Inv2[2][2];

float p_1[2]={0,0};


double u[2];

double u1[2];

double u2[2];

double M_1[2];

double B_1[2];

double K_1[2];

double a_1[2];

double torque[2];

double vv1[2];

double rms_of_cart_velocity;

double rms_of_force;

void setup() {

    Serial.begin(19200);

    Serial.print("100.1");

    Serial.print("\t\t");

    Serial.print("120.2");

    Serial.print("\t\t");

    Serial.print("0");

    Serial.print("\t\t");

    Serial.println("0");

    //Serial.println("Start Communication");
```

```
setup_sample_timer();  
setup_motors();  
  
// Activate internal pull-ups for all encoder pins  
bitSet(PORTA, 0); // Arduino pin No. 22  
bitSet(PORTA, 1); // Arduino pin No. 23  
bitSet(PORTA, 2); // Arduino pin No. 24  
bitSet(PORTA, 3); // Arduino pin No. 25  
bitSet(PORTA, 4); // Arduino pin No. 26  
bitSet(PORTA, 5); // Arduino pin No. 27  
  
// Activate internal pull-ups for all limit switches  
bitSet(PORTC, 7); // Arduino pin No. 30  
bitSet(PORTC, 6); // Arduino pin No. 31  
  
//power to encoders  
pinMode(52, OUTPUT); //power to encoder 1  
pinMode(53, OUTPUT);  
digitalWrite(52, HIGH);  
digitalWrite(53, LOW);  
pinMode(51, OUTPUT); //power to encoder 2  
pinMode(50, OUTPUT);  
digitalWrite(51, HIGH);  
digitalWrite(50, LOW);  
  
temp_store_2_double.val1 = 0;  
temp_store_2_double.val2 = 0;  
  
motor_actuate(1, 70);  
motor_actuate(2, 120);
```

```
while (limit_flag[0] == 0 || limit_flag[1] == 0)
{
    if (!bitRead(PINC, 7))
    {
        motor_actuate(1, 0);
        limit_flag[0] = 1;
    }
    if (!bitRead(PINC, 6))
    {
        motor_actuate(2, 0);
        limit_flag[1] = 1;
    }
}

motor_actuate(1, 0);
motor_actuate(2, 0);
encoder_pos[0] = 3449;
encoder_pos[1] = 4253;

curr_encoder_pos[0] = encoder_pos[0]; //saved because it is volatile
curr_encoder_pos[1] = encoder_pos[1]; //saved because it is volatile
curr_motor_pos_deg[0] = (curr_encoder_pos[0] * 90) / 3000; //converting current encoder_1 position
to degrees
curr_motor_pos_deg[1] = (curr_encoder_pos[1] * 90) / 3000; //converting current encoder_2 position
to degrees

temp_store_2_double = forward_kine(curr_motor_pos_deg[0] , curr_motor_pos_deg[1]);
ref_x[0] = temp_store_2_double.val1;
ref_x[1] = temp_store_2_double.val2;
```

```
}

double time_new = 0;
unsigned long int time_old = 0;
double th;
double R;
int movement_speed = 8000;

void loop() {
    //updating global target
    //if (allow_sampling_commands == 1)
    //{
        //allow_sampling_commands = 0;
        // local_cart_target[0] = 0.40;
        // local_cart_target[1] = 0.40;

    //}

    //getting all sensor inputs
    if (allow_sampling_input == 1)
    {
        allow_sampling_input = 0;
        force_1 = get_value_FSR_1(); //taking average value of FSR_1
        force_2 = get_value_FSR_2(); //taking average value of FSR_2
        force_3 = get_value_FSR_3(); //taking average value of FSR_3
    }
    Serial.println("Actual Force 1");
```



```
Serial.println(force_1);

Serial.println("Actual Force 2");

Serial.println(force_2);

Serial.println("Actual Force 3");

Serial.println(force_3);


curr_encoder_pos[0] = encoder_pos[0]; //saved because it is volatile
curr_encoder_pos[1] = encoder_pos[1]; //saved because it is volatile

curr_motor_pos_deg[0] = (curr_encoder_pos[0] * 90) / 3000; //converting current encoder_1 position
to degrees

curr_motor_pos_deg[1] = (curr_encoder_pos[1] * 90) / 3000; //converting current encoder_2 position
to degrees


//////////calculating motor velocities

curr_encoder_vel[0] = encoder_vel[0] * 20;          //(vel/50) * 1000
curr_encoder_vel[1] = encoder_vel[1] * 20;          //(vel/50) * 1000

curr_motor_vel_deg[0] = (curr_encoder_vel[0] * 90) / 3000;
curr_motor_vel_deg[1] = (curr_encoder_vel[1] * 90) / 3000;

//////////calculating motor accelration

curr_encoder_acc[0] = encoder_acc[0] * 20;          //(vel/50) * 1000
curr_encoder_acc[1] = encoder_acc[1] * 20;          //(vel/50) * 1000

curr_motor_acc_deg[0] = (curr_encoder_acc[0] * 90) / 3000;
curr_motor_acc_deg[1] = (curr_encoder_acc[1] * 90) / 3000;


if (allow_printing_curr_pos == 1)
{
    allow_printing_curr_pos = 0;


    rms_of_cart_velocity = sqrt(curr_cart_velocity[0]*curr_cart_velocity[0] +
curr_cart_velocity[1]*curr_cart_velocity[1]);
```

```
rms_of_force = sqrt(F[0]*F[0] + F[1]*F[1]);
Serial.print(curr_motor_pos_deg[0]);
Serial.print("\t\t");
Serial.print(curr_motor_pos_deg[1]);
Serial.print("\t\t");
Serial.print(rms_of_force);
Serial.print("\t\t");
Serial.println(rms_of_cart_velocity);

// Serial.print(local_cart_target[0] * 20);
// Serial.print("\t\t");
// Serial.println(local_cart_target[1] * 20);

//applying jacobian
temp_store_2_double = jacobian(curr_motor_pos_deg[0] , curr_motor_pos_deg[1] ,
curr_motor_vel_deg[0] , curr_motor_vel_deg[1]);//(th1,th2,thdot1,thdot2)
curr_cart_velocity[0] = temp_store_2_double.val1;
curr_cart_velocity[1] = temp_store_2_double.val2;
//acceleration
temp_store_2_double = jacobi_acc(curr_motor_pos_deg[0] , curr_motor_pos_deg[1] ,
curr_motor_vel_deg[0] ,
curr_motor_vel_deg[1],curr_motor_acc_deg[0],curr_motor_acc_deg[1]);//(th1,th2,thdot1,thdot2,thdot
dot1,thdotdot2)
curr_cart_acceleration[0] = temp_store_2_double.val1;
curr_cart_acceleration[1] = temp_store_2_double.val2;
//applying forward kinematics
temp_store_2_double = forward_kine(curr_motor_pos_deg[0] , curr_motor_pos_deg[1]);
```

```

curr_cart_position[0] = temp_store_2_double.val1;
curr_cart_position[1] = temp_store_2_double.val2;

//applying inverse jacobian

// temp_store_2_double = inv_jacobian(curr_motor_pos_deg[0] , curr_motor_pos_deg[1] ,
global_target_vector[0] , global_target_vector[1]); //(th1,th2,xdot,ydot)

// curr_motor_vel_deg_from_jacobian[0] = temp_store_2_double.val1;
// curr_motor_vel_deg_from_jacobian[1] = temp_store_2_double.val2;

//applying inverse kinematics

// temp_store_2_double = inverse_kine(local_cart_target[0],local_cart_target[1]);
// target_encoder_pos[0] = temp_store_2_double.val1;
// target_encoder_pos[1] = temp_store_2_double.val2;

//applying jacobian transpose

// temp_store_2_double = jacobian_transpose(curr_motor_pos_deg[0] , curr_motor_pos_deg[1] ,
jin[0] , jin[1]);

// Ta[0] = temp_store_2_double.val1 / 18000;
// Ta[1] = temp_store_2_double.val2 / 18000;

//making force vectors
if ((force_1 - force_3) > 1) //true when force_1 is dominant
{
    theta_force_1 = curr_motor_pos_deg[0] + curr_motor_pos_deg[1] - 90; //calculating theta in degrees
    for force vector_1

    vector_force_1[0] = cos(deg2rad(theta_force_1)) * (force_1 - force_3); //x_coordinate of force
    vector_1

    vector_force_1[1] = sin(deg2rad(theta_force_1)) * (force_1 - force_3); //y_coordinate of force
    vector_1
}

```

```
else
{
    vector_force_1[0] = 0;
    vector_force_1[1] = 0;
}
if (force_2 > 1)
{
    theta_force_2 = curr_motor_pos_deg[0] + curr_motor_pos_deg[1] + 180; //calculating theta in
degrees for force vector_2
    vector_force_2[0] = cos(deg2rad(theta_force_2)) * force_2; //x_coordinate of force vector_2
    vector_force_2[1] = sin(deg2rad(theta_force_2)) * force_2; //y_coordinate of force vector_2
}
else
{
    vector_force_2[0] = 0;
    vector_force_2[1] = 0;
}
if ((force_3 - force_1) > 1) //true when force_3 is dominant
{
    theta_force_3 = curr_motor_pos_deg[0] + curr_motor_pos_deg[1] + 90;
    vector_force_3[0] = cos(deg2rad(theta_force_3)) * (force_3 - force_1);
    vector_force_3[1] = sin(deg2rad(theta_force_3)) * (force_3 - force_1);
}
else
{
    vector_force_3[0] = 0;
    vector_force_3[1] = 0;
}
```

```
////////////////////////////////operation mode selection////////////////////////////////
```

```
// if(operation_mode == 2)
// {
//   time_old = millis();
//   time_new = time_old % movement_speed;
//   th = (time_new/movement_speed) * 360;
//   R = 0.08;
//   local_cart_target[0] = R*cosd(th) + 0.2125 ;
//   local_cart_target[1] = 0.2125 ;
// }
// if(operation_mode == 3)
// {
//   time_old = millis();
//   time_new = time_old % movement_speed;
//   th = (time_new/movement_speed) * 360;
//   R = 0.08;
//   local_cart_target[0] = 0.2125 ;
//   local_cart_target[1] = R*sind(th) + 0.2125 ;
// }
```

```
//////////////////////////////// new impedance control
////////////////////////////////
//Dynamic Variable.
```

```
q[0] = curr_motor_pos_deg[0];
q[1] = curr_motor_pos_deg[1];
q_dot[0] = curr_motor_vel_deg[0];
```

```

q_dot[1] = curr_motor_vel_deg[1];
//Km[0] = input_Km[0] * 10;
//Km[1] = input_Km[1] * 10;
//Bm[0] = input_Bm[0];
//Bm[1] = input_Bm[1];

////////// extra temporary testing stuff //////////
//q[0] = 89;
//q[1] = 91;
//q_dot[0] = 0;
//q_dot[1] = 0;
//temp_store_2_double = forward_kine(q[0] , q[1]);
//curr_cart_position[0] = temp_store_2_double.val1;
//curr_cart_position[1] = temp_store_2_double.val2;
//temp_store_2_double = jacobian(q[0] , q[1] , q_dot[0] , q_dot[1]); //(th1,th2,thdot1,thdot2)
//curr_cart_velocity[0] = temp_store_2_double.val1;
//curr_cart_velocity[1] = temp_store_2_double.val2;
////////////////////////////////////

p1 = m1 * (pow(Lc1, 2)) + m2 * (pow(L1, 2)) + l1;
p2 = m2 * (pow(Lc2, 2)) + l2;
p3 = m2 * L1 * Lc2;
p4 = (m1 * Lc1) + (m2 * L1);
p5 = m2 * Lc2;

/*
p1 = -p1; //reverse M and l
p2 = -p2;
p3 = -p3;

```

```

p4 = -p4;
p5 = -p5;*/
// Km[0] = -4; //reverse Km and Bm
// Km[1] = -4;
// Bm[0] = -2;
// Bm[1] = -2;

```

```

// Inverse of half of identity matrix

```

```

IMm[0][0] = Mm[1][1] / MMm;

```

```

IMm[0][1] = Mm[0][1] / MMm;

```

```

IMm[1][0] = Mm[1][0] / MMm;

```

```

IMm[1][1] = Mm[0][0] / MMm;

```

```

IKm[0][0] = Km[1][1] / Kmm;

```

```

IKm[0][1] = Km[0][1] / Kmm;

```

```

IKm[1][0] = Km[1][0] / Kmm;

```

```

IKm[1][1] = Km[0][0] / Kmm;

```

```

// External Force

```

```

F[0] = (vector_force_1[0] + vector_force_2[0] + vector_force_3[0]) ;

```

```

F[1] = (vector_force_1[1] + vector_force_2[1] + vector_force_3[1]) ;

```

```

F[0]=constrain(F[0],-3,3);

```

```

F[1]=constrain(F[1],-3,3);

```

```

//a_1[0]=F[0]-(Bm[0]*(xd_dot[0]-curr_cart_velocity[0]))-(Km[0]*(xd[0]-curr_cart_position[0]));

```

```

//a_1[1]=F[1]-(Bm[1]*(xd_dot[1]-curr_cart_velocity[1]))-(Km[1]*(xd[1]-curr_cart_position[1]));

```

```

//xd_dotdot[0]=IMm[0][0]*a_1[0]+IMm[0][1]*a_1[1];
//xd_dotdot[1]=IMm[1][0]*a_1[0]+IMm[1][1]*a_1[1];
//now=millis();
//xd_dot[0]=(xd_dot[0]+xd_dotdot[0]*(now-last_time_sampled))*conver;
//xd_dot[1]=(xd_dot[1]+xd_dotdot[1]*(now-last_time_sampled))*conver;
//xd[0]=(xd[0]+xd_dot[0]*(now-last_time_sampled))*conver;
//xd[1]=(xd[1]+xd_dot[1]*(now-last_time_sampled))*conver;
//last_time_sampled=now;

//M_1[0]=(Mm[0][0]*(curr_cart_acceleration[0])+Mm[0][1]*curr_cart_acceleration[1]);
//M_1[1]=(Mm[1][0]*(curr_cart_acceleration[0])+Mm[1][1]*curr_cart_acceleration[1]);
B_1[0]=(Bm[0][0]*(prev_delta_x_dot[0])+(Bm[0][1]*prev_delta_x_dot[1]));
B_1[1]=(Bm[1][0]*(prev_delta_x_dot[0])+(Bm[1][1]*prev_delta_x_dot[1]));
K_1[0]=(Km[0][0]*(prev_delta_x[0])+(Km[0][1]*prev_delta_x[1]));
K_1[1]=(Km[1][0]*(prev_delta_x[0])+(Km[1][1]*prev_delta_x[1]));
a_1[0]=F[0]-B_1[0]-K_1[0];
a_1[1]=F[1]-B_1[1]-K_1[1];
delta_x_dotdot[0]=IMm[0][0]*a_1[0]+IMm[0][1]*a_1[1];
delta_x_dotdot[1]=IMm[1][0]*a_1[0]+IMm[1][1]*a_1[1];

now=millis();
Ts=(now-last_time_sampled);
Ts=Ts/2000;
Ts=0.082;

p_1[0]=delta_x_dotdot[0]+prev_delta_x_dotdot[0];
p_1[1]=delta_x_dotdot[1]+prev_delta_x_dotdot[1];

```



```
Serial.print("p_1");  
Serial.print("\t\t");  
Serial.print(Ts*p_1[0]);  
Serial.print("\t\t");  
Serial.println(Ts*p_1[1]);  
  
delta_x_dot[0]=(Ts*p_1[0]+prev_delta_x_dot[0]);  
delta_x_dot[1]=(Ts*p_1[1]+prev_delta_x_dot[1]);  
delta_x[0]=(((Ts)*(delta_x_dot[0]+prev_delta_x_dot[0]))+prev_delta_x[0])/1;  
delta_x[1]=(((Ts)*(delta_x_dot[1]+prev_delta_x_dot[1]))+prev_delta_x[1])/1;  
  
/* if(delta_x[0]>0.03){  
    delta_x[0]=0.03;  
}  
if(delta_x[1]>0.03){  
    delta_x[1]=0.03;  
}  
if(delta_x[0]<-0.03){  
    delta_x[0]=-0.03;  
}  
if(delta_x[1]<-0.03){  
    delta_x[1]=-0.03;  
}  
*/  
  
//last_time_sampled=now;  
Serial.print("Time elapsed");  
Serial.print("\t\t");
```

```
Serial.print(last_time_sampled);
```

```
Serial.print("\t\t");
```

```
Serial.print(now);
```

```
Serial.print("\t\t");
```

```
Serial.println(Ts,5);
```

```
last_time_sampled=now;
```

```
Serial.print("previous End_effector_position: ");
```

```
Serial.print("\t\t");
```

```
Serial.print(prev_delta_x[0],5);
```

```
Serial.print("\t\t");
```

```
Serial.println(prev_delta_x[1],5);
```

```
Serial.print("Previous End_effector_velocity: ");
```

```
Serial.print("\t\t");
```

```
Serial.print(prev_delta_x_dot[0]);
```

```
Serial.print("\t\t");
```

```
Serial.println(prev_delta_x_dot[1]);
```

```
Serial.print("Previous End_effector_acceleration: ");
```

```
Serial.print("\t\t");
```

```
Serial.print(prev_delta_x_dotdot[0]);
```

```
Serial.print("\t\t");
```

```
Serial.println(prev_delta_x_dotdot[1]);
```

```
prev_delta_x[0]=delta_x[0];
```

```
prev_delta_x[1]=delta_x[1];
```

```
prev_delta_x_dot[0]=delta_x_dot[0];
```

```
prev_delta_x_dot[1]=delta_x_dot[1];
```

```
prev_delta_x_dotdot[0]=delta_x_dotdot[0];
```

```
prev_delta_x_dotdot[1]=delta_x_dotdot[1];
```

```
actual_x[0]=ref_x[0]+delta_x[0];
actual_x[1]=ref_x[1]+delta_x[1];

//torque[0]=Kp[0]*(xd[0]-curr_cart_position[0])-(Kd[0]*curr_cart_velocity[0]);
//torque[1]=Kp[1]*(xd[1]-curr_cart_position[1])-(Kd[1]*curr_cart_velocity[1]);
//if (torque[0] < 5 && torque[0] > -5)
//{
// torque[0] = 0;
//}
//if (torque[1] < 5 && torque[1] > -5)
//{
// torque[1] = 0;
//}
// motor_target_vel[0] = motor_1_vel_2_pwm(torque[0]/6);
// motor_target_vel[1] = motor_2_vel_2_pwm(torque[1]);
temp_store_2_double = inverse_kine(actual_x[0],actual_x[1]);
qd[0]=temp_store_2_double.val1;
qd[1]=temp_store_2_double.val2;
Serial.print("End_effector_position: ");
Serial.print("\t\t");
Serial.print(delta_x[0],5);
Serial.print("\t\t");
Serial.println(delta_x[1],5);
Serial.print("End_effector_velocity: ");
Serial.print("\t\t");
Serial.print(delta_x_dot[0]);
Serial.print("\t\t");
Serial.println(delta_x_dot[1]);
Serial.print("End_effector_acceleration: ");
```

```
Serial.print("\t\t");
Serial.print(delta_x_dotdot[0]);
Serial.print("\t\t");
Serial.println(delta_x_dotdot[1]);
Serial.print("commanded theta: ");
qd[0]=qd[0]*90/3000;
qd[1]=qd[1]*90/3000;

Serial.print("\t\t");
Serial.print(qd[0]);
Serial.print("\t\t");
Serial.println(qd[1]);
Serial.print("actual theta: ");
Serial.print("\t\t");
Serial.print(q[0]);
Serial.print("\t\t");
Serial.println(q[1]);
Serial.print("Force: ");
Serial.print("\t\t");
Serial.print(F[0]);
Serial.print("\t\t");
Serial.println(F[1]);
Serial.print("actual: ");
Serial.print("\t\t");
Serial.print(actual_x[0]);
Serial.print("\t\t");
Serial.println(actual_x[1]);
motor_target_vel[0]=Kp[0]*(qd[0]-q[0])+Kd[0]*q_dot[0];
motor_target_vel[1]=Kp[1]*(qd[1]-q[1])+Kd[1]*q_dot[1];
```

```
torque[0]= motor_1_vel_to_pwm(motor_target_vel[0]);
torque[1]= motor_2_vel_2_pwm(motor_target_vel[1]);

Serial.print("q dot:");
Serial.print("\t\t");
Serial.print(q_dot[0]);
Serial.print("\t\t");
Serial.println(q_dot[1]);
Serial.print("PWM:");
Serial.print("\t\t");
Serial.print(motor_1_vel_2_pwm(motor_target_vel[0]));
Serial.print("\t\t");
Serial.println(motor_2_vel_2_pwm(motor_target_vel[1]));
Serial.print("Position:");
Serial.print("\t\t");
Serial.print(curr_cart_position[0]);
Serial.print("\t\t");
Serial.println(curr_cart_position[1]);
Serial.println("-----");
motor_actuate(1, -100);
motor_actuate(2, -120);

/*
if (qd[0]>q[0])
{ motor_actuate(1,70);
}
if(qd[1]>q[1])
{ motor_actuate(2,-100);
```

```
    }
    if (qd[0]<q[0])
    { motor_actuate(1,-70);
    }
    if(qd[1]<q[1])
    { motor_actuate(2,120);
    }

    if (q[0]<=qd[0]+2&&q[0]>=qd[0]-2)
    { motor_actuate(1,0);
    Serial.println("lkjghkjfhcvbcxxdfm,mhvm");
    }
    if(q[1]<=qd[1]+2&&q[1]>=qd[1]-2)
    { motor_actuate(2,0);
    Serial.println("lkjgh,mhvm");
    }
    */
    /*

    Serial.print("commanded theta: ");
    Serial.println(qd[0]);
    Serial.println(qd[1]);
    Serial.print("actual theta.....: ");
    Serial.println(q[0]);
    Serial.println(q[1]);*/
    // To actuate the motors correspondingly
    // motor_actuate(1, -80);
    // motor_actuate(2, -100);
    }}
```

```
//Actuate the motors with provided pwm
void motor_actuate(int Motor_num,int Motor_pwm)
{
    if(Motor_num == 1)
    {
        if(Motor_pwm>0 && encoder_pos[0] < 3200)
        {
            digitalWrite(In1,HIGH);
            digitalWrite(In2,LOW);
            curr_neg_dir[0] = 0;
        }
        else if(Motor_pwm<0 && encoder_pos[0] > -3200)
        {
            digitalWrite(In1,LOW);
            digitalWrite(In2,HIGH);
            curr_neg_dir[0] = 1;
        }
        else
        {
            digitalWrite(In1,LOW);
            digitalWrite(In2,LOW);
        }
        analogWrite(EnA,abs(Motor_pwm));
    }
    else if(Motor_num == 2)
    {
        if(Motor_pwm>0 && encoder_pos[1] < 4000)
        {
            digitalWrite(In3,LOW);
```

```
digitalWrite(In4,HIGH);
curr_neg_dir[1] = 0;
}
//else if(Motor_pwm<0 && encoder_pos[1] > -4000)
else if(Motor_pwm<0 && encoder_pos[1] > 500)
{
digitalWrite(In3,HIGH);
digitalWrite(In4,LOW);
curr_neg_dir[1] = 1;
}
else
{
digitalWrite(In3,LOW);
digitalWrite(In4,LOW);
}
analogWrite(EnB,abs(Motor_pwm));
}
}

/// Forward Kinematics, Inverse Kinematics, Jacobian, Jacobian Transpose, Jacobian Inverse

double deg2rad(double x)
{
double y;
y = (x*pi)/180;
return y;
}

double rad2deg(double x)
{
```



```
double y;  
y = (x*180)/pi;  
return y;  
}
```

```
double sind(double x)  
{  
    double y;  
    y = deg2rad(x);  
    y = sin(y);  
    return y;  
}
```

```
double cosd(double x)  
{  
    double y;  
    y = deg2rad(x);  
    y = cos(y);  
    return y;  
}
```

```
store_2_double inverse_kine(double x,double y)  
{  
    store_2_double angles_of_motors;  
  
    double temp = 0;  
    double cosh2;  
    double sinh2;
```

```
double th2;

double th1;

double k1;

double k2;

double centreangle;

double xsq = pow(x,2);

double ysq = pow(y,2);

double r1sq = pow(L1,2);

double r2sq = pow(L2,2);

double costh2_denom = 2*L1;

costh2_denom = costh2_denom * L2;

double costh2_num = xsq + ysq - r1sq - r2sq;

costh2 = costh2_num / costh2_denom;


sinh2 = sqrt(1-(costh2*costh2));

th2 = (atan2((sinh2),(costh2)))*180/pi;

k1 = L1 + (L2*costh2);

k2 = L2*sinh2;

centreangle = (atan2(abs(y),abs(x)))*180/pi;


if (y < 0 && x >= 0)
{
centreangle = -1*centreangle;
}

else if (y<0 && x<0)
{
centreangle = centreangle - 180;
}

else if (y>=0 && x<0)
```

```
{
temp = 90 - centreangle;
centreangle = 90 + temp;
}

th1 = centreangle - (atan2(k2,k1)*180/pi);

if(th1>180)
{
th1 = th1 - 360;
}
else if(th1<-180)
{
th1 = th1 + 360;
}

angles_of_motors.val1 = (th1*3000)/90;
angles_of_motors.val2 = (th2*3000)/90;

// angles_of_motors.val1 = constrain(angles_of_motors.val1,-3300,3300);
//angles_of_motors.val2 = constrain(angles_of_motors.val2,-4100,4100);

return angles_of_motors;
}

store_2_double forward_kine(double th1,double th2)
{
store_2_double xy;
xy.val1 = (L1*cosd(th1)) + (L2*cosd(th1 + th2));
```

```
xy.val2 = (L1*sind(th1)) + (L2*sind(th1 + th2));  
return xy;  
}
```

```
store_2_double jacobian(double th1,double th2,double thdot1,double thdot2)
```

```
{  
    store_2_double vel;  
    double th12 = th1 + th2;  
    double j11;  
    double j12;  
    double j21;  
    double j22;  
    double xdot1;  
    double xdot2;  
    double ydot1;  
    double ydot2;  
    j22 = L2*cosd(th12);  
    j12 = -L2*sind(th12);  
  
    j11 = (-L1*sind(th1)) + j12;  
    j21 = (L1*cosd(th1)) + j22;  
  
    xdot1 = j11*thdot1;  
    xdot2 = j12*thdot2;  
    ydot1 = j21*thdot1;  
    ydot2 = j22*thdot2;  
  
    vel.val1 = xdot1 + xdot2;  
    vel.val2 = ydot1 + ydot2;
```

```

    return vel;
}

store_2_double jacobi_acc(double th1,double th2,double thdot1,double thdot2,double
thdotdot1,double thdotdot2)
{
    store_2_double acc;
    double th12 = th1 + th2;
    double thdot12=thdot1+thdot2;
    double thdotdot12=thdotdot1+thdotdot2;
    double xdotdot1;
    double xdotdot2;
    double ydotdot1;
    double ydotdot2;
    double xdotdot;
    double ydotdot;
    xdotdot1 = (-L1*sind(th1)*thdotdot1)-(thdot1*thdot1*L1*cosd(th1));
    xdotdot2 = (L2*sind(th12)*thdotdot12)+(thdot12*thdot12*L2*cosd(th12));
    xdotdot=xdotdot1-xdotdot2;
    ydotdot1 = (L1*cosd(th1)*thdotdot1)-(thdot1*thdot1*L1*sind(th1));
    ydotdot2 = (L2*cosd(th12)*thdotdot12)-(thdot12*thdot12*L2*sind(th12));
    ydotdot=ydotdot1+ydotdot2;
    acc.val1 = xdotdot;
    acc.val2 = ydotdot;
    return acc;
}

store_2_double jacobian_transpose(double th1,double th2,double thdot1,double thdot2)
{
    store_2_double vel;

```

```
double th12 = th1 + th2;
double j11;
double j12;
double j21;
double j22;
double xdot1;
double xdot2;
double ydot1;
double ydot2;
j12 = -L2*sind(th12);
j22 = L2*cosd(th12);

j11 = (-L1*sind(th1)) + j12;
j21 = (L1*cosd(th1)) + j22;

xdot1 = j11*thdot1;
xdot2 = j21*thdot2;
ydot1 = j12*thdot1;
ydot2 = j22*thdot2;

vel.val1 = xdot1 + xdot2;
vel.val2 = ydot1 + ydot2;

return vel;
}
```

```
store_2_double inv_jacobian(double th1,double th2,double xdot,double ydot)
{
    store_2_double vel;
    double th12 = th1 + th2;
    double j11;
    double j12;
    double j21;
    double j22;
    double det;
    j11 = L2*cosd(th12);
    j12 = L2*sind(th12);

    j21 = (-L1*cosd(th1)) - j11;
    j22 = (-L1*sind(th1)) - j12;

    det = L1 * L2 * sind(th2);

    vel.val1 = ((j11*xdot) + (j12*ydot)) / det;
    vel.val2 = ((j21*xdot) + (j22*ydot)) / det;
    return vel;
}

/// Communication Functions to communicate with matlab gui in various modes ///
```

```
float string_to_float(String str)
{
    float ans;
    int str_len = str.length() + 1;
```

```
char char_array[str_len];
str.toCharArray(char_array, str_len);
ans = atof(char_array);
return ans;
}

volatile boolean encoder[2][2]={0};          // 2 encoders, each having 2 channels
//volatile double x[2][2]={0},u[2][2]={0};    // To save 2 recent samples for 2 encoders
//double Kp[2]={20,15},Kd[2]={0},Ki[2]={0};    // PID Gains for the 3 motors
const double T=0.0001;                      // Sampling Time

ISR(TIMERS5_OVF_vect)
{
    TCNT5=63936; //reset timer-5 again to the period of 100 micro sec

    boolean a,b;
    static char tally_for_sampling_input = 0;
    static char tally_for_sampling_commands = 0;
    static char tally_for_velocity_transition = 0;
    static int tally_for_encoder_velocity_calculation = 0;
    static int tally_for_allow_printing_curr_pos = 0;
    static int prev_encoder_pos[2] = {0};
    static int prev_encoder_vel[2] = {0};

    // keeping time for when to get inputs from sensors and commands
    tally_for_sampling_input++;
    tally_for_sampling_commands++;
    tally_for_velocity_transition++;
    tally_for_encoder_velocity_calculation++;
    tally_for_allow_printing_curr_pos++;
```



```
if(tally_for_sampling_input == 50) //5ms interval
{
    tally_for_sampling_input = 0;
    allow_sampling_input = 1;
}

if(tally_for_sampling_commands == 100) //10ms interval
{
    tally_for_sampling_commands = 0;
    allow_sampling_commands = 1;
}

if(tally_for_velocity_transition == 20) //2ms interval
{
    tally_for_velocity_transition = 0;
    allow_velocity_transition = 1;
}

if(tally_for_encoder_velocity_calculation == 500) //50ms interval
{
    tally_for_encoder_velocity_calculation = 0;
    encoder_vel[0] = encoder_pos[0] - prev_encoder_pos[0];
    encoder_vel[1] = encoder_pos[1] - prev_encoder_pos[1];
    encoder_acc[0] = encoder_vel[0] - prev_encoder_vel[0];
    encoder_acc[1] = encoder_vel[1] - prev_encoder_vel[1];
    prev_encoder_pos[0] = encoder_pos[0];
    prev_encoder_pos[1] = encoder_pos[1];
    prev_encoder_vel[0] = encoder_vel[0];
    prev_encoder_vel[1] = encoder_vel[1];
}

if(tally_for_allow_printing_curr_pos == 1100) //110ms interval
```

```
{
    tally_for_allow_printing_curr_pos = 0;
    allow_printing_curr_pos = 1;
}
for(byte i=0;i<2;i++)
{
    if(i==0)
    {
        a=bitRead(PINA,0);
        b=bitRead(PINA,1);
    }
    else if(i==1)
    {
        b=bitRead(PINA,2);
        a=bitRead(PINA,3);
    }
    if(encoder[i][0]!=a || encoder[i][1]!=b)
    {
        encoder_read(a,b,i);
        encoder[i][0]=a;
        encoder[i][1]=b;
    }
}
}
```

```
void encoder_read(boolean a,boolean b,byte encdr_no)
```

```
{
    int QEM[4][4]={0,-1,1,2},{1,0,2,-1},{-1,2,0,1},{2,1,-1,0}}; //Quadrature Encoder Matrix
```

```
char direct;

static byte dir[2]={0};

byte dir_old;

dir_old=dir[encdr_no];
dir[encdr_no]=(2*a)+b;      // new value of dir[encdr_no]
direct=QEM[dir_old][dir[encdr_no]]; // represent direction

if(direct==1)
    encoder_pos[encdr_no]++;    // count total no. of changes (original frequency of encoder)
else if(direct== -1)
    encoder_pos[encdr_no]--;
}

double motor_1_vel_2_pwm(double x)
{
    double y;

    if (x>-5 && x<5)
    {
        y = 0;
    }
    else if(x>5)
    {
        if(x>150)
        {
            y = 150 + pwm_offset_m1;
        }
        else
```

```
{  
    y = x + pwm_offset_m1 + 5;  
}  
}  
else  
{  
    if(x<-150&& x<-5)  
    {  
        y = -150 - pwm_offset_m1;  
    }  
    else  
    {  
        y = x - pwm_offset_m1;  
    }  
}  
  
return y;  
}  
  
double motor_2_vel_2_pwm(double x)  
{  
    double y;  
  
    if (x>-5&& x<5)  
    {  
        y = 0;  
    }  
    else if(x>5)  
    {
```

```
    if(x>150)
    {
        y = 150 + pwm_offset_m2;
    }
    else
    {
        y = x + pwm_offset_m2;
    }
}
else
{
    if(x<-150&& x<-5)
    {
        y = -150 - pwm_offset_m2;
    }
    else
    {
        y = x - pwm_offset_m2;
    }
}
return y;
}
```

```
double motor_1_vel_to_pwm(double x)
{
    double y;
    Serial.println("chor");
    if (x==0)
```

```
{
    Serial.println("choooooor");
    y = 0;
}

if(x>=150){
    y = 150 + pwm_offset_m1;
    Serial.println("coooooor");
}

if(x<150&& x>0){
    y = x + pwm_offset_m1 + 5;
    Serial.println("coor");
}

if(x<=-150){
    y = -150 - pwm_offset_m1;
}

if(x>-150&& x<0){
    y = x - pwm_offset_m1 ;
}

return y;
}

double get_value_FSR_1()
{
    static double dataset_forcesensor[samplesize_forcesensor_1];
    static int count_forcesensor;
    double ans = 0;
    //ans = analogRead(A1);
    dataset_forcesensor[count_forcesensor] = analogRead(A1);
    for(int i = 0; i < samplesize_forcesensor_1; i++)
    {
```

```
    ans = ans + dataset_forcesensor[i];
}
ans = ans / samplesize_forcesensor_1;
count_forcesensor++;
if(count_forcesensor == samplesize_forcesensor_1)
{
    count_forcesensor = 0;
}
return ans;
}

double get_value_FSR_2()
{
    static double dataset_forcesensor[samplesize_forcesensor_2];
    static int count_forcesensor;
    double ans = 0;
    //ans = analogRead(A1);
    dataset_forcesensor[count_forcesensor] = analogRead(A2);
    for(int i = 0; i < samplesize_forcesensor_2; i++)
    {
        ans = ans + dataset_forcesensor[i];
    }
    ans = ans / samplesize_forcesensor_2;
    count_forcesensor++;
    if(count_forcesensor == samplesize_forcesensor_2)
    {
        count_forcesensor = 0;
    }
    return ans;
}
```

```
}

double get_value_FSR_3()
{
    static double dataset_forcesensor[samplesize_forcesensor_3];
    static int count_forcesensor;
    double ans = 0;
    //ans = analogRead(A1);
    dataset_forcesensor[count_forcesensor] = analogRead(A3);
    for(int i = 0; i < samplesize_forcesensor_3; i++)
    {
        ans = ans + dataset_forcesensor[i];
    }
    ans = ans /samplesize_forcesensor_3;
    count_forcesensor++;
    if(count_forcesensor == samplesize_forcesensor_3)
    {
        count_forcesensor = 0;
    }
    return ans;
}

///// Timer-5 interrupt is being used for sensors' readings (software interrupt) /////

void setup_sample_timer()
{
    TCNT5=63936; // Overflow after every 100 microsec
    TCCR5A=0;
    TCCR5B=1<<CS50;
    bitSet(TIMSK5,TOIE5); // enable overflow interrupt
```



```
}
```

```
void setup_motors()
```

```
{
```

```
    pinMode(In1,OUTPUT);
```

```
    pinMode(In2,OUTPUT);
```

```
    pinMode(In3,OUTPUT);
```

```
    pinMode(In4,OUTPUT);
```

```
    pinMode(EnA,OUTPUT);
```

```
    pinMode(EnB,OUTPUT);
```

```
    digitalWrite(In1,LOW);
```

```
    digitalWrite(In2,LOW);
```

```
    digitalWrite(In3,LOW);
```

```
    digitalWrite(In4,LOW);
```

```
    digitalWrite(EnA,LOW);
```

```
    digitalWrite(EnB,LOW);
```

```
}
```